LAPORAN TUGAS BESAR

ARSITEKTUR APLIKASI TERDISTRIBUSI

MILESTONE 1

OwenDISTRIBUTED



Disusun oleh:

SekreDagoSuites

Daniel Pedrosa Wu	13523099
Steven Owen Liauw	13523103
Haegen Quinston	13523109

TEKNIK INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **2**
DESKRIPSI UMUM SISTEM **3**
ARSITEKTUR SISTEM **4**
DAFTAR TEKNOLOGI **8**
ASUMSI **10**
PETUNJUK PENGGUNAAN SISTEM **12**
Problem 1 — Arsitektur dan Interoperabilitas Data **13**
Rancangan Solusi 13
Alasan Pemilihan dan Trade-Off 14
Batasan Solusi 14
Pengujian dan Bukti Penyelesaian 15
Mekanisme Evolusi Skema dan Cakupan Perubahan yang Didukung 19
Problem 2 — Concurrency, Availability, & Graceful Degradation **21**
Rancangan Solusi 21
Alasan Pemilihan dan Trade-Off 22
Batasan Solusi 23
Pengujian dan Bukti Penyelesaian 24
Model Konkurensi, Penanganan Kegagalan, dan Konfigurasi Load Testing 30
Problem 3 — Service-to-Service Authentication & Trust **32**
Rancangan Solusi 32
Alasan Pemilihan dan Trade-Off 33
Batasan Solusi 33
Pengujian dan Bukti Penyelesaian 34
Mekanisme Autentikasi dan Analisis Keamanan 36
Problem 4 — Arsitektur Microservice dan Flexible Storage **38**
Rancangan Solusi 38
Alasan Pemilihan dan Trade-Off 39
Batasan Solusi 41
Pengujian dan Bukti Penyelesaian 41
Dasar Pembagian Service dan Perbandingan Penyimpanan 44
Problem 5 — Fan-out Event Distribution/Pub-sub **46**
Rancangan Solusi 46
Alasan Pemilihan dan Trade-Off 47
Batasan Solusi 47
Pengujian dan Bukti Penyelesaian 48
Pemilihan Broker, Semantik Pengiriman, dan Idempotensi Consumer 51
LAMPIRAN **53**

DESKRIPSI UMUM SISTEM

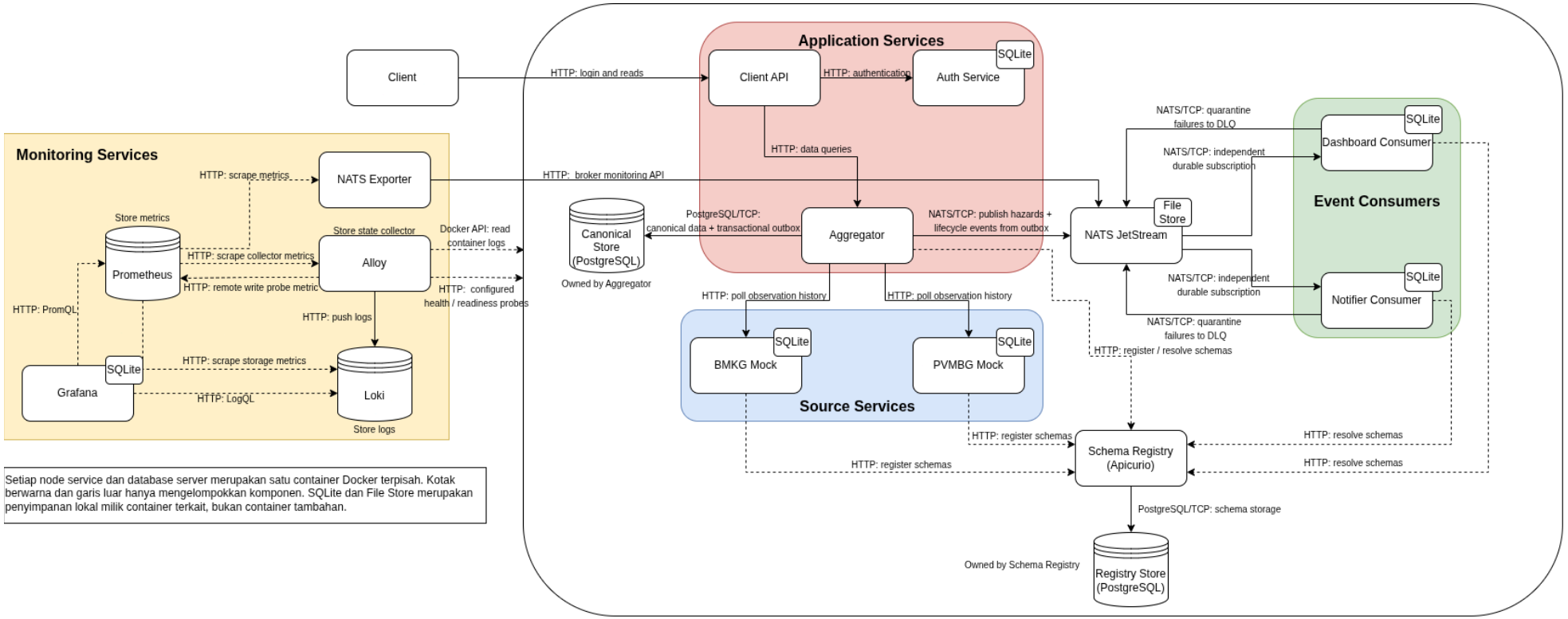
Sistem yang dikembangkan merupakan platform koordinasi kebencanaan yang menempatkan BNPB sebagai *aggregator* antara dua instansi sumber, yaitu BMKG sebagai penyedia data gempa bumi dan tsunami, serta PVMBG sebagai penyedia data gunung api. Kedua instansi tersebut memiliki format data, waktu respons, dan mekanisme autentikasi yang berbeda. BNPB bertugas mengambil data dari kedua instansi, menormalisasikannya ke dalam satu format baku bernama HazardEvent, menyimpannya, serta menyajikannya kepada *downstream client* sesuai dengan hak akses masing-masing.

Alur kerja sistem diawali oleh Aggregator yang melakukan *polling* terhadap kedua *mock* dengan interval beberapa detik, memetakan data yang diperoleh ke dalam format HazardEvent, kemudian menyimpannya ke dalam Canonical Store. Setiap HazardEvent baru juga diterbitkan ke *message broker* sehingga pihak lain dapat berlangganan tanpa Aggregator perlu mengetahui identitas pelanggannya. *Client* tidak pernah mengakses penyimpanan secara langsung. Seluruh permintaan *client* masuk melalui Client API, yang memverifikasi token ke Auth Service sebelum meminta data ke Aggregator.

Source of truth dalam tugas besar ini adalah *mock* dari dua instansi, yaitu BMKG untuk data seismik dan PVMBG untuk data vulkanik, yang diimplementasikan sesuai dengan kontrak pada spesifikasi. Terdapat tiga downstream client, yaitu Media, Tim Lapangan, dan BNPB Internal, yang seluruhnya memiliki akses baca terhadap data kebencanaan.

Secara keseluruhan, Stack utama terdiri atas sebelas container komponen aplikasi dan infrastruktur. Lima container monitoring dijalankan melalui profile monitoring sehingga jumlah keseluruhannya menjadi enam belas container. Client TUI, consumer demonstrasi tambahan, dan broker-init dijalankan melalui profile Docker Compose sesuai kebutuhan.

ARSITEKTUR SISTEM



Sistem menggunakan arsitektur *service* terdistribusi untuk mengambil data kebencanaan, mengintegrasikannya ke format kanonis, menyediakan akses kepada *client*, dan mendistribusikan perubahan melalui *event*. Tanggung jawab tersebut dipisahkan ke dalam Client API, Auth Service, Aggregator, dua *service* sumber, *broker*, dan dua *consumer*. Pengelolaan *schema* serta *monitoring* dijalankan oleh komponen tersendiri agar setiap bagian memiliki fungsi dan kepemilikan data yang jelas.

Setiap *service* dan *server database* PostgreSQL dijalankan dalam *container* Docker terpisah. Beberapa *service* memanfaatkan SQLite sebagai *database* di dalam *container*-nya masing-masing sehingga tidak memerlukan *container* database tambahan yang terpisah. Data persisten ditempatkan pada *volume* masing-masing, termasuk *source archive, authentication session*, data kanonis, pesan *broker*, hasil pemrosesan *consumer*, dan penyimpanan *monitoring*. *Dispatcher outbox* berjalan sebagai *goroutine* di dalam proses Aggregator dan tidak memiliki container tersendiri.

Client mengakses sistem melalui Client API menggunakan HTTP. Client API melayani *login*, *refresh token*, *logout*, pembacaan *hazard*, dan pembacaan laporan awal atau *notice*. Operasi autentikasi diteruskan kepada Auth Service melalui HTTP. Pada request yang dilindungi, Client API juga memvalidasi *access token* melalui *endpoint* `POST /internal/introspect`. Auth Service menyimpan sesi dan *hash token* pada SQLite miliknya, sedangkan identitas *client* beserta kredensialnya berasal dari konfigurasi *service*. Client API menggunakan *scope* hasil autentikasi untuk menentukan akses terhadap tampilan ringkasan atau data mentah.

BMKG Mock menyediakan data gempa dan peringatan tsunami, sedangkan PVMBG Mock menyediakan laporan aktivitas vulkanik. Masing-masing *service* memiliki SQLite untuk menyimpan observasi serta menyediakan *endpoint history* melalui HTTP. Aggregator menjalankan *polling* data terkini dan pembacaan *history* secara terpisah. Pembacaan riwayat menggunakan *cursor* yang disimpan di PostgreSQL sehingga pengambilan dapat dilanjutkan dari posisi sebelumnya setelah proses terhenti. Pada konfigurasi Compose, pengambilan data terkini dan *history* menggunakan Avro melalui HTTP. Endpoint sumber juga menyediakan respons JSON untuk permintaan yang tidak meminta format Avro.

Data yang diterima Aggregator divalidasi, dipetakan, dan dideduplikasi sebelum disimpan dalam Canonical Store berbasis PostgreSQL. Aggregator juga mengelola *notice*, revisi data, status sumber, dan posisi pembacaan riwayat. Yang dimaksud dengan *notice* adalah catatan laporan awal dari sumber yang belum dapat dipetakan menjadi data hazard karena masih menunggu informasi pendukung. Contohnya, laporan gempa yang berpotensi tsunami, tetapi pemeriksaan peringatan tsunami terkait belum selesai. Canonical Store hanya diakses langsung oleh Aggregator. Client API membaca data melalui *endpoint*

internal Aggregator sehingga akses pengguna tetap mengikuti antarmuka layanan yang disediakan. Proses pembacaan ini menggunakan data yang telah tersimpan dan tidak menunggu *polling* sumber selesai pada setiap permintaan.

Publikasi *event* menggunakan pola *transactional outbox*. Aggregator menyimpan perubahan data beserta catatan publikasi tertunda dalam transaksi PostgreSQL yang sama. *Dispatcher* internal kemudian menerbitkan *event* ke NATS JetStream. Catatan *outbox* dihapus setelah publikasi memperoleh konfirmasi dari *broker*. Jika publikasi gagal atau transaksi penyelesaiannya terputus, catatan tetap tersedia untuk dicoba kembali. Sistem memisahkan *stream hazard* dan *lifecycle*. *Stream hazard* membawa data *hazard* awal, sedangkan *stream lifecycle* membawa perubahan berikutnya, termasuk revisi *hazard*, perubahan *notice*, dan status sumber.

NATS JetStream menyimpan *stream hazard*, *stream lifecycle*, dan *dead-letter queue* atau DLQ menggunakan *file storage* pada *volume broker*. Dashboard Consumer dan Notifier Consumer berkomunikasi dengan *broker* dan memiliki *durable subscription* untuk *hazard*, serta *subscription* tersendiri untuk *lifecycle*. Dengan demikian, posisi pemrosesan setiap *consumer* dan setiap jenis *stream* dapat dipertahankan secara independen. Pesan tertunda dapat diproses kembali selama masih tersedia sesuai *retention policy*.

Dashboard Consumer menyimpan proyeksi *hazard*, *notice*, dan status sumber pada SQLite miliknya. Notifier Consumer menyimpan proyeksi serta catatan notifikasi untuk perubahan *hazard* atau *notice* yang diterapkan. Kedua *consumer* menyediakan *endpoint* HTTP `GET /records` untuk memeriksa hasil pemrosesan. *Endpoint* tersebut terpisah dari jalur pembacaan *client* melalui Client API dan Aggregator.

Consumer menyimpan hasil pemrosesan dan informasi deduplikasi dalam satu transaksi SQLite sebelum mengirim ACK yang dikonfirmasi *broker*. Informasi deduplikasi mencakup identitas *stream*, *sequence* pesan, dan *digest* isi pesan. Untuk *event lifecycle*, *consumer* juga memeriksa identitas *event* dan nomor revisi agar pengiriman ulang atau *event* lama tidak menerapkan perubahan yang sama maupun menimpa versi yang lebih baru. Mekanisme ini mencegah pesan yang dikirim ulang menerapkan perubahan yang sama lebih dari sekali, serta mencegah event lama menimpa data yang lebih baru.

Kegagalan sementara ditangani melalui retry dengan penundaan. Pesan tertentu yang bermasalah dipindahkan ke DLQ, sedangkan masalah yang menunjukkan ketidakcocokan schema, gangguan penyimpanan serius, atau banyak kegagalan beruntun dapat menghentikan pemrosesan sampai operator menangani penyebabnya. DLQ berfungsi sebagai tempat penyimpanan terpisah bagi pesan bermasalah

agar dapat diperiksa, dianalisis, dan diproses ulang secara manual setelah penyebab kegagalan ditangani, tanpa mengganggu pemrosesan pesan lainnya. *Consumer* memastikan pesan telah berhasil disimpan dalam DLQ sebelum mengakui pesan asal.

Pemrosesan yang dihentikan dapat dilanjutkan oleh operator melalui perintah lokal consumer resume yang dijalankan menggunakan `docker compose exec` pada container consumer terkait setelah penyebab kegagalan ditangani. Perintah tersebut berkomunikasi dengan proses consumer melalui Unix socket lokal dengan akses terbatas berdasarkan izin filesystem. Dalam satu service consumer, *subscription hazard* dan *lifecycle* menggunakan kendali penghentian yang sama sehingga *recovery* berlaku untuk keduanya. *Recovery* mempertahankan hasil pemrosesan, informasi deduplikasi, cache schema, dan riwayat retry, serta tidak memproses ulang atau menghapus pesan dalam DLQ.

Schema Registry berbasis Apicurio menyediakan pengelolaan *schema* melalui HTTP. BMKG Mock dan PVMBG Mock mendaftarkan *schema* observasi, sedangkan Aggregator mengambil *schema* sumber serta mendaftarkan *schema event hazard* dan *lifecycle*. *Consumer* mengambil *schema* berdasarkan ID dalam pesan Avro dan menyimpan *cache schema* pada SQLite masing-masing. Schema Registry menyimpan datanya pada PostgreSQL terpisah. NATS JetStream tidak langsung berinteraksi dengan *registry* sehingga pembentukan dan pembacaan pesan menjadi tanggung jawab *service* pengirim dan penerima.

Monitoring dijalankan oleh Alloy, NATS Exporter, Prometheus, Loki, dan Grafana. Alloy memeriksa *endpoint health/readiness* yang dikonfigurasi melalui HTTP serta membaca *log container* melalui Docker API pada Unix *socket*. Metrik pemeriksaan dikirim ke Prometheus melalui HTTP *remote write*, sementara *log* dikirim ke Loki melalui HTTP. NATS Exporter membaca API *monitoring broker* melalui HTTP dan menyediakan metrik untuk diambil Prometheus. Prometheus juga mengambil metrik dirinya sendiri, Alloy, dan Loki.

Grafana mengakses Prometheus dan Loki melalui HTTP menggunakan PromQL dan LogQL untuk menyajikan *dashboard* serta mengevaluasi aturan *alert*. Prometheus memiliki *volume* metrik, Loki memiliki *volume log*, Alloy menyimpan *state collector*, dan Grafana menggunakan *volume* lokal untuk konfigurasi serta *database* SQLite. Komponen aplikasi tidak memanggil layanan *monitoring* dalam jalur pemrosesannya sehingga gangguan *monitoring* tidak secara langsung menghentikan pengambilan data maupun pemrosesan *event*.

DAFTAR TEKNOLOGI

Komponen	Teknologi	Alasan pemilihan
Seluruh *service* yang dikembangkan	Go 1.27.1	Penggunaan satu bahasa pemrograman untuk seluruh *service* mempermudah *code review* lintas *service*. Goroutine juga sesuai untuk dua kebutuhan utama sistem, yaitu *polling* terhadap dua sumber secara independen dan penanganan banyak permintaan secara konkuren. Hasil *build* berupa satu *static binary* sehingga ukuran *image container* tetap kecil.
HTTP *server*	Gin	Gin digunakan pada mock, Aggregator, Client API, dan Auth Service untuk routing dan middleware. Kedua consumer menggunakan pustaka standar net/http untuk endpoint `/health`, `/ready`, dan `/records`. Recovery dijalankan melalui perintah lokal consumer resume.
Logging	log/slog dengan keluaran JSON	Tersedia pada pustaka standar Go dan mendukung structured log dalam format JSON. Middleware dan kode pencatatan aplikasi menambahkan *correlation ID* serta latensi *request* dan panggilan keluar.
Canonical Store	PostgreSQL 16 dengan kolom JSONB, diakses melalui pgx v5	Atribut dinamis dapat disimpan tanpa migrasi skema, sementara dukungan transaksi dan *unique index* tetap tersedia untuk keperluan *outbox* dan deduplikasi. Penjelasan lebih lanjut terdapat pada bagian P4.
Message broker	NATS JetStream 2.15 dengan klien nats.go	Dipilih karena ringan untuk dijalankan pada lingkungan lokal, yaitu cukup dengan satu *binary* dan satu berkas konfigurasi. JetStream menyediakan *persistent stream*, *durable consumer*, dan *explicit acknowledgement* yang dibutuhkan untuk menerapkan semantik *at-least-once*.
Mekanisme evolusi skema	Avro dengan Apicurio Registry 3.3 dan pustaka goavro v2	Mendukung perubahan schema yang kompatibel tanpa mengubah kode penerima, sesuai kebijakan reader schema. Registry menerapkan pemeriksaan FULL_TRANSITIVE saat registrasi, sedangkan Aggregator melakukan validasi data dan aturan tambahan.

Penyimpanan *Auth Service*, *Mock* dan *Consumer*	SQLite, satu berkas untuk setiap *service*	Setiap *service* terkait memiliki penyimpanannya sendiri tanpa memerlukan *database server* tambahan. Data *session* dan data *consumer* berukuran kecil sehingga SQLite sudah memadai.
Downstream client	Aplikasi terminal (TUI) berbasis Go	Memadai untuk mendemonstrasikan tiga identitas *client*, penolakan akses terhadap data mentah, serta alur *refresh token* tanpa perlu membangun antarmuka web.
Pengemasan dan orkestrasi	Docker dan Docker Compose	Setiap *service* memiliki satu Dockerfile, dan seluruh sistem dapat dijalankan dengan satu perintah sesuai dengan ketentuan implementasi.
Load testing	k6, dijalankan sebagai *container*	Tidak memerlukan instalasi pada mesin lokal, skenario pengujian dapat ditulis sebagai skrip, dan hasilnya langsung melaporkan p50, p95, p99, *throughput*, serta *error rate*.
Unit testing	Pustaka testing bawaan Go dan testify	Memadai untuk menguji logika pemetaan, aturan penentuan *severity*, serta perilaku *handler*.
Monitoring	Grafana, Prometheus, Loki, Alloy, dan NATS exporter.	Grafana menyajikan dashboard dan pencarian log, Prometheus menyimpan metrik, Loki menyimpan log, Alloy menjalankan pemeriksaan endpoint dan mengumpulkan log Docker, NATS exporter menyediakan metrik broker tanpa mengonsumsi event.

ASUMSI

Berikut adalah asumsi yang digunakan selama proses perancangan dan implementasi sistem.

1. Parameter *since* yang tidak disertakan atau tidak dapat di-*parse* dianggap tidak memberikan batas waktu. Endpoint observasi *mock* mengembalikan seluruh dataset live yang masih tersedia. Untuk *timestamp* yang valid, batas waktu bersifat inklusif.
2. Pada `GET /tsunami-warnings`, since diartikan sebagai batas waktu kejadian gempa terkait, yaitu *occurred_at* pada *SeismicEvent*. Kontrak *TsunamiWarning* tidak menyediakan timestamp penerbitan warning yang dapat dijadikan dasar filter.
3. Spesifikasi memberikan *related_event_id* sebagai referensi dari *TsunamiWarning* ke *SeismicEvent*. Pada data demo, satu *SeismicEvent* diasumsikan memiliki nol atau satu TsunamiWarning terkait (one-to-zero-or-one). Karena itu, field warning terkait disimpan langsung dalam attributes.
4. Jika sumber mengirim lebih dari satu warning untuk gempa yang sama, implementasi tetap menentukan severity berdasarkan tingkat ancaman tertinggi dari seluruh warning terkait yang sudah diterima dan tersimpan pada saat penilaian. Spesifikasi memprioritaskan Awas, tetapi tidak merinci pemilihan antara beberapa warning lain; pemilihan tingkat tertinggi merupakan kebijakan untuk menangani kondisi di luar asumsi data demo. Warning tidak harus diterima bersamaan, dan kedatangan warning berikutnya memicu penilaian ulang. Ketentuan bahwa warning hanya digunakan untuk potential_tsunami=true tetap mengikuti spesifikasi.
5. Identifier pada kontrak dianggap unik dalam entitas sumbernya, bukan harus unik secara global lintas instansi. Hazard dikenali melalui pasangan *source* dan *source_ref_id*, sehingga ID yang sama dari BMKG dan PVMBG tetap menjadi dua hazard berbeda. Penerimaan ulang pasangan yang sama mempertahankan *hazard_id*, isi kanonik yang berubah memperbarui snapshot dan revision, sedangkan isi yang sama tidak menghasilkan hazard baru.
6. Ketiga identitas ditetapkan sebelum startup melalui konfigurasi Auth Service.

PETUNJUK PENGGUNAAN SISTEM

Petunjuk lengkap ada di README.md pada *root* repositori. Ringkasnya seperti berikut.

1. Siapkan Docker dan Docker Compose.

2. Siapkan konfigurasi berdasarkan `.env.example`. Salin bagian konfigurasi root ke `.env`, lalu buat `.env` pada direktori *service* sesuai dengan pembagian variabel.

3. Dari *root* repositori, jalankan `docker compose up -d --build`. Untuk menjalankan aplikasi sekaligus lima container monitoring, gunakan perintah berikut: `docker compose --profile monitoring up -d --build`

4. Periksa status menggunakan `docker compose ps` dan tunggu hingga seluruh container layanan yang dijalankan berstatus *healthy*.

5. Untuk mencoba sebagai *client*, jalankan `docker compose run --rm --build client-tui` lalu pilih identitas yang ingin dipakai.

Port host ditentukan melalui `.env` root. Konfigurasi contoh menggunakan port 18080 untuk Client API, 18081 untuk BMKG, 18082 untuk PVMBG, dan 13000 untuk Grafana. Schema Registry, broker NATS, API monitoring NATS, serta Prometheus, Loki, Alloy, dan NATS Exporter diakses melalui jaringan internal Docker. Port internal Client API, BMKG, dan PVMBG pada konfigurasi contoh masing-masing adalah 8080, 8081, dan 8082, serta dapat diubah melalui konfigurasi. Cara menjalankan setiap skenario pengujian tersedia pada bagian Demo scenarios dalam README. Urutan pengujian lengkap beserta hasil yang diharapkan dijelaskan dalam `Test Runbook.md`

Problem 1 - Arsitektur dan Interoperabilitas Data

Rancangan Solusi

Permasalahan utama pada prototipe awal adalah proses *parsing* yang bergantung pada struktur data yang diasumsikan tetap. Untuk mengatasinya, dilakukan pemisahan terhadap dua tanggung jawab yang sebelumnya tercampur, yaitu mekanisme pembacaan data dari sumber dan mekanisme pemetaan data ke format baku.

Pembacaan data dilakukan menggunakan Avro dengan *schema registry*. Secara bawaan, kedua *mock* tetap mengembalikan respons dalam format JSON sesuai dengan kontrak. Apabila permintaan menyertakan *header* Accept dengan nilai application/avro, *mock* mengembalikan data dalam format Avro yang diawali oleh satu *magic byte* dan empat *byte* ID skema. Aggregator meminta Avro untuk *polling live*, sedangkan *replay* melalui `/history` menggunakan JSON. Aggregator mengambil *writer schema* berdasarkan ID pada respons, lalu membacanya menggunakan *reader schema* aktif. *Reader schema* awal dimuat dari berkas yang ditentukan oleh BMKG_SEISMIC_READER_SCHEMA, BMKG_TSUNAMI_READER_SCHEMA, dan PVMBG_READER_SCHEMA. Dengan demikian, *writer schema* dari registry dan *reader schema* yang dipilih Aggregator memiliki peran yang berbeda.

Pemetaan data dilakukan secara dinamis setelah validasi dan resolusi *writer schema* terhadap *reader schema*. Aggregator mengambil field yang diperlukan untuk mengisi field baku, misalnya report_id, volcano_id, dan alert_level. Field yang diterima oleh reader aktif dan tidak memiliki padanan kanonik diteruskan ke attributes setelah normalisasi nilai Avro. Field yang tidak terdapat pada *reader* aktif diabaikan. Jalur *ingest* tidak memiliki cabang pemetaan khusus untuk confidence_level. Aturan validasi tambahan, seperti rentang nilai, dijalankan oleh Aggregator. Pemeriksaan FULL_TRANSITIVE pada registry memeriksa kompatibilitas Avro dan tidak menggantikan validasi semantik tersebut.

Ketika `POST /admin/schema-version` dipanggil, sumber mengaktifkan *writer* v2 yang menambahkan confidence_level sebagai optional field setelah registrasi yang diperiksa dengan FULL_TRANSITIVE. Polling live Aggregator menemukan writer schema yang berbeda dan menyimpan kandidat reader kedua jika kompatibel dua arah dengan baseline. Penemuan kandidat tidak mengubah *reader* aktif. Selama *reader* v1 aktif, laporan tetap diproses dan confidence_level diabaikan. Operator kemudian mengaktifkan kandidat melalui `POST /admin/schema-version` pada Aggregator dengan token admin tersendiri. Endpoint Aggregator mengembalikan 409 apabila kandidat belum ditemukan; helper demo mencoba kembali 409 dengan batas waktu 120 detik. History memakai *reader* aktif yang

sama dan tidak menjadi jalur penemuan kandidat. Laporan yang diterima setelah *reader* v2 aktif menyimpan confidence_level di attributes tanpa *restart* atau r*ebuild service*.

Agar perubahan skema tidak berdampak pada komponen *downstream*, dua hal lain turut dijaga. Pertama, Canonical Store menyimpan attributes sebagai kolom JSONB sehingga tidak diperlukan penambahan kolom. Kedua, pada pesan yang dikirim ke *broker*, attributes dikodekan sebagai *map* berisi nilai JSON sehingga skema pesan *broker* tidak berubah ketika sumber menambahkan *field* baru.

Alasan Pemilihan dan Trade-Off

Terdapat dua pendekatan yang dipertimbangkan yaitu

1. JSON dengan kebijakan *additive-only*
   JSON mudah diperiksa secara langsung, tetapi perilaku field tak dikenal ditentukan oleh *decoder* dan pemetaan aplikasi, bukan oleh format JSON itu sendiri. Validasi kontrak dan pemeriksaan kompatibilitas perlu ditambahkan agar perubahan nama atau tipe field tidak diterima tanpa penilaian.
2. Avro dengan *schema registry*
   Skema disertakan bersama data melalui ID skema sehingga penerima dapat membaca *field* baru tanpa perubahan kode. *Registry* juga menolak skema yang tidak kompatibel pada saat pendaftaran, sehingga kesalahan terdeteksi di sisi sumber sebelum data dikirimkan.

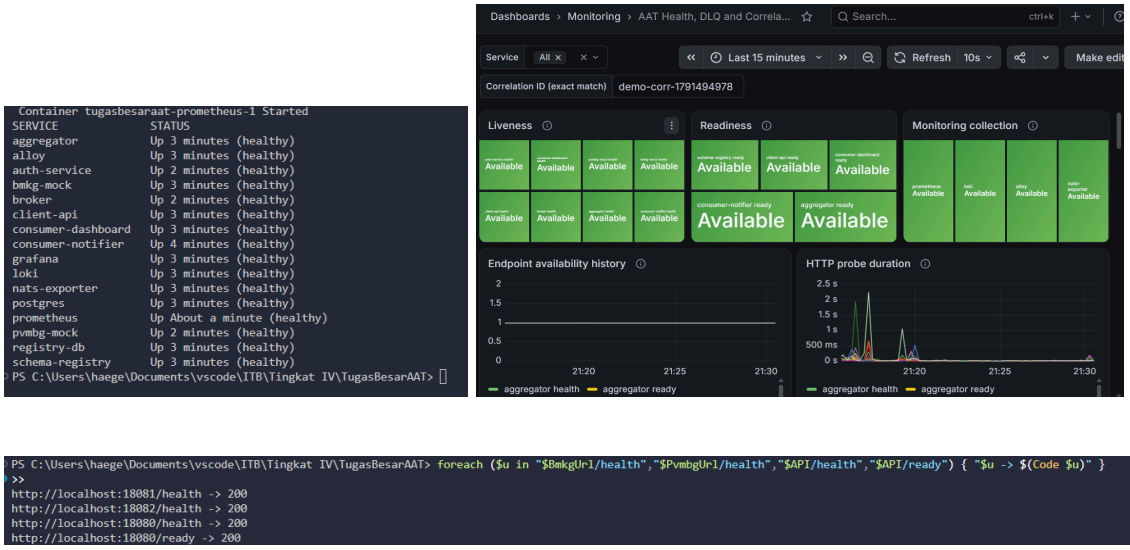
Konsekuensi dari pendekatan ini adalah adanya komponen tambahan yang harus dijalankan dan dipelihara, yaitu *registry* beserta *database*-nya. Konsekuensi tersebut dapat diterima karena manfaatnya menjawab inti permasalahan secara langsung. Untuk mengurangi ketergantungan terhadap *registry*, setiap *service* menyimpan skema yang pernah digunakan di dalam *in-memory cache*, sedangkan consumer menyimpannya pada SQLite masing-masing, sehingga kegagalan *registry* yang bersifat sementara tidak menghentikan alur data yang sedang berjalan.

Batasan Solusi

1. *Registry* menjadi titik ketergantungan baru. Apabila *registry* tidak tersedia ketika Aggregator baru dijalankan dan belum memiliki skema di *cache*, proses *polling* akan gagal hingga *registry* kembali tersedia. Data yang telah tersimpan tetap dapat dibaca.

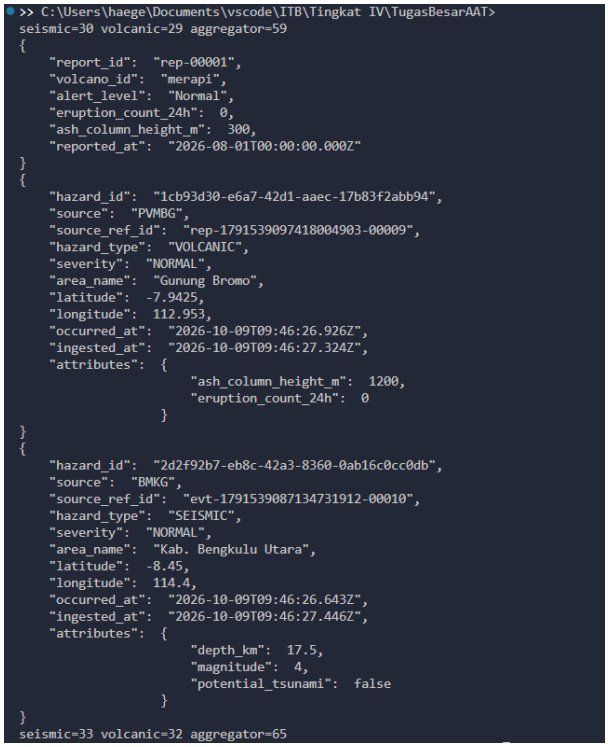
2. Apabila terdapat *required field* yang hilang atau memiliki tipe yang salah pada satu *record*, seluruh hasil *polling* tersebut ditolak. Pendekatan ini dipilih secara sengaja untuk menghindari penyimpanan data parsial yang berpotensi tidak valid.
3. *Record* yang sama diperbarui secara *in-place* pada *polling* berikutnya. Akibatnya, apabila skema PVMBG dikembalikan ke versi sebelumnya, *confidence_level* turut hilang dari baris yang masih dikirimkan oleh sumber. Tabel *hazard_events* menyimpan snapshot kanonik terbaru. Riwayat observasi sumber tersedia pada arsip mock, sedangkan perubahan material dan transisi pelaporan dicatat pada *report_lifecycle_events*. Keduanya digunakan untuk pemulihan dan penelusuran perubahan.
4. *Optional record field* bernilai *null* dihilangkan dari attributes.

Pengujian dan Bukti Penyelesaian

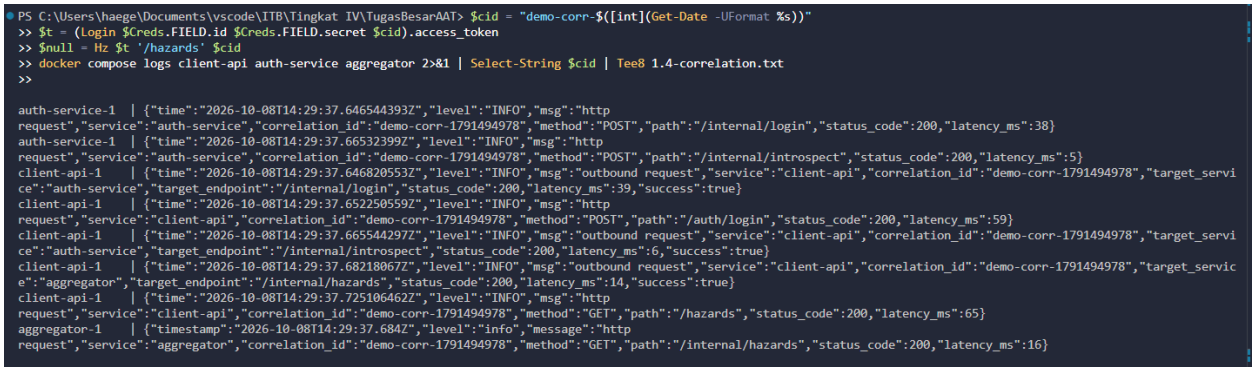


Gambar P1.1. Status container dan pemeriksaan health serta readiness saat inisialisasi.

Pengujian diawali dengan menjalankan sistem melalui Docker Compose dan memeriksa kesiapan layanan. Gambar P1.1 menunjukkan container berstatus healthy serta respons HTTP 200 dari endpoint health dan readiness yang diperiksa. Setelah sistem siap, data pada skema awal diamati untuk memastikan bahwa Aggregator dapat menghasilkan HazardEvent dari kedua instansi. Pada Gambar P1.2, keluaran kanonik PVMBG menggunakan hazard_type VOLCANIC dengan atribut ash_column_height_m dan eruption_count_24h, sedangkan keluaran BMKG menggunakan hazard_type SEISMIC dengan atribut magnitude, depth_km, dan potential_tsunami. Contoh BMKG memiliki magnitude 5,4 tanpa potensi tsunami dan menghasilkan severity WASPADA, sesuai aturan pemetaan pada spesifikasi.

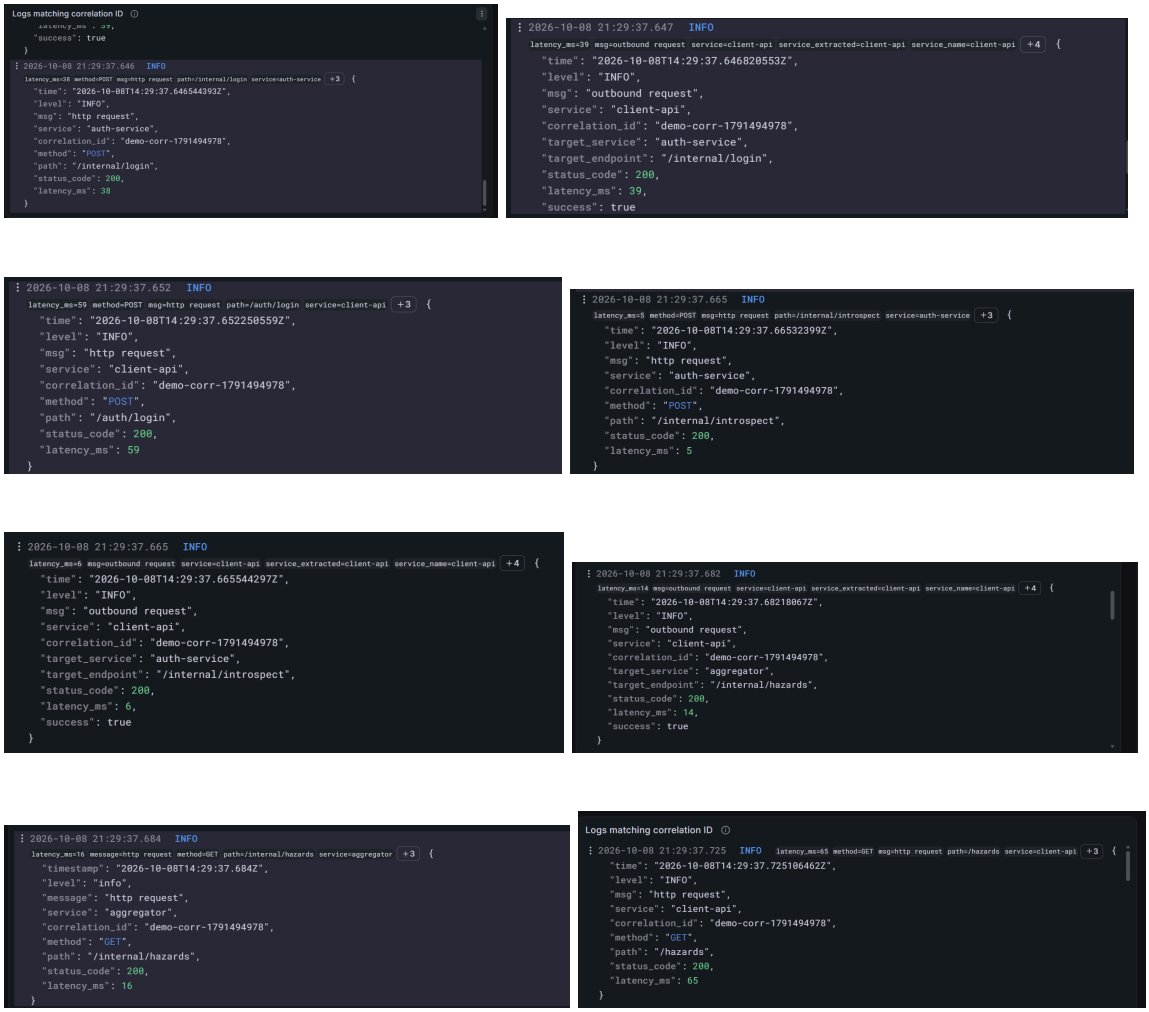


Gambar P1.2. Contoh data sumber PVMBG dan HazardEvent awal dari kedua instansi.

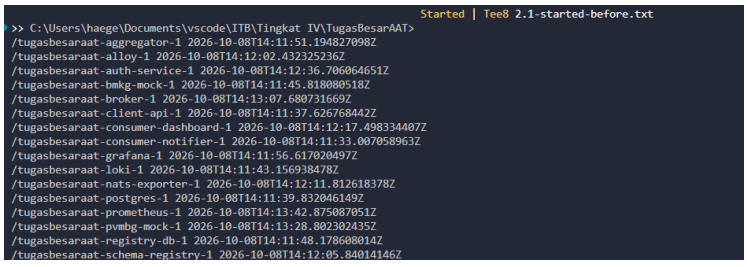


Gambar P1.3. Alur autentikasi dan pembacaan data dengan correlation ID yang sama.

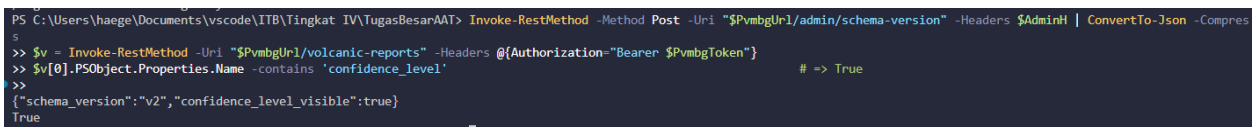
Penelusuran permintaan selanjutnya dilakukan menggunakan correlation ID demo-corr-1791494978. Gambar P1.3 dan P1.4 memperlihatkan ID yang sama pada alur autentikasi, verifikasi token, dan pembacaan data melalui Client API menuju Aggregator. Log tersebut mencatat endpoint, status respons, dan latensi panggilan, sehingga hubungan antarservice dalam satu alur permintaan dapat ditelusuri.



Gambar P1.4. Penelusuran log lintas service pada Grafana.

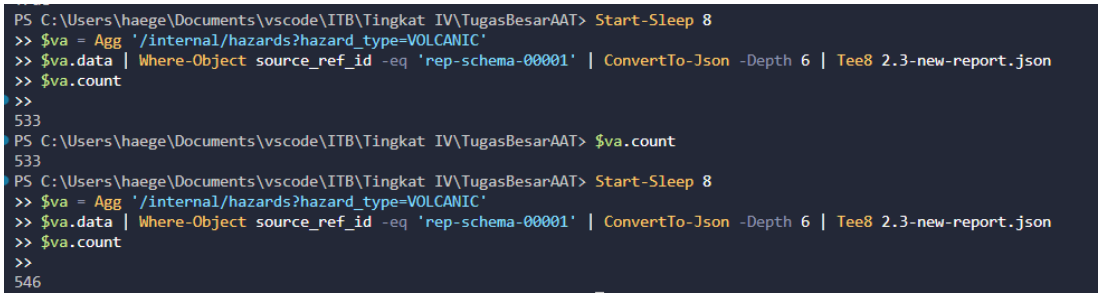


Gambar P1.5. Pencatatan waktu mulai container sebelum perubahan skema.

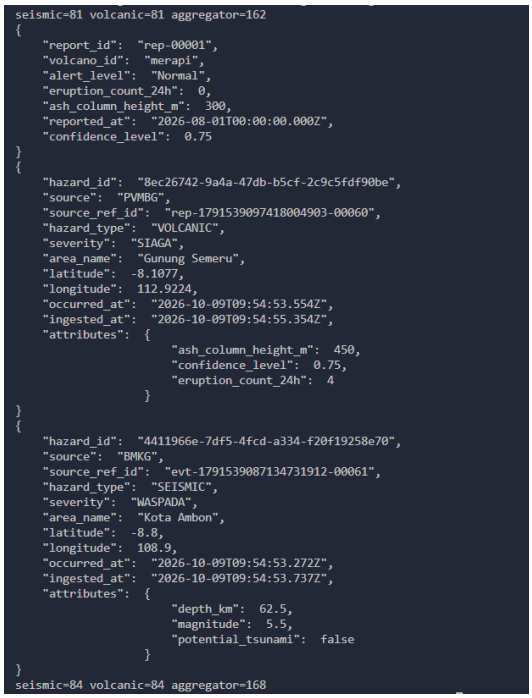


Gambar P1.6. Aktivasi skema PVMBG v2 dan kemunculan confidence_level pada sumber.

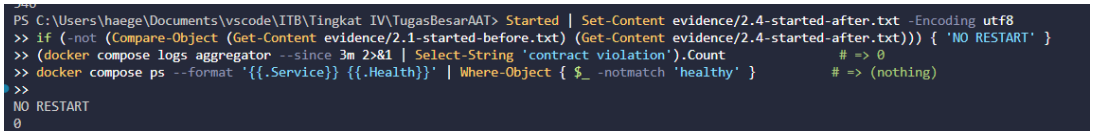
Perubahan skema dipicu ketika sistem masih berjalan. Waktu mulai container dicatat sebagai pembanding pada Gambar P1.5, kemudian POST /admin/schema-version dipanggil pada PVMBG. Gambar P1.6 menunjukkan schema_version bernilai v2 dan confidence_level_visible bernilai true, disertai pemeriksaan kemunculan confidence_level pada respons sumber. Setelah perubahan, jumlah record vulkanik yang terbaca melalui Aggregator meningkat dari 533 menjadi 546 pada Gambar P1.7. Bukti ini dilengkapi oleh Gambar P1.8, yang memperlihatkan confidence_level bernilai 0,75 di dalam attributes pada HazardEvent vulkanik. Dengan demikian, Aggregator tetap menghasilkan data kanonik dan meneruskan field tambahan sesuai kebijakan yang dinyatakan.



Gambar P1.7. Pertambahan record vulkanik setelah perubahan skema.



Gambar P1.8. Kemunculan confidence_level pada respons PVMBG dan attributes HazardEvent.



Gambar P1.9. Pemeriksaan restart, status container, dan log contract violation.

Pemeriksaan akhir pada Gambar P1.9 menghasilkan NO RESTART, tidak menemukan log contract violation pada interval yang diperiksa, dan tidak menampilkan container dengan status di luar healthy. Hasil tersebut menunjukkan bahwa perubahan skema berlangsung tanpa restart yang terdeteksi. Bersama keluaran kanonik pada Gambar P1.8, bukti ini mendukung bahwa field tambahan dapat diterima dan disimpan dalam attributes tanpa menghentikan pemrosesan data.

Mekanisme Evolusi Skema dan Cakupan Perubahan yang Didukung

Kebijakan yang diterapkan adalah sebagai berikut: *field* yang tidak terdapat pada *reader schema* aktif diabaikan, sedangkan *field* yang terdapat pada *reader schema* aktif tetapi tidak memiliki padanan kanonik diteruskan ke dalam *attributes*. Kebijakan ini berlaku untuk BMKG maupun PVMBG. Dengan aturan yang sama, confidence_level diabaikan selama *reader* v1 aktif dan diteruskan ke *attributes* setelah *reader* v2 diaktifkan, tanpa penanganan khusus. Pada sisi *Client* API, respons untuk Media disusun berdasarkan daftar *field* ringkasan yang didefinisikan secara eksplisit, sehingga *field* baru pada *attributes* tidak dapat terekspos kepada Media.

Mekanisme yang digunakan merupakan gabungan dari *schema registry*, evolusi skema Avro, dan pemetaan *field* secara dinamis pada Aggregator. Perubahan skema yang didukung adalah penambahan *optional field* baru. *Field* tersebut dapat terbaca tanpa *restart* dan tersimpan di attributes setelah *reader schema* yang memuatnya diaktifkan oleh operator.

Perubahan skema yang secara sadar tidak didukung adalah sebagai berikut.

1. Penghapusan atau penggantian nama *field* yang dibutuhkan dalam proses pemetaan, misalnya *report_id* atau *alert_level*.
2. Perubahan tipe data pada *required field*, misalnya *eruption_count_24h* yang berubah menjadi tipe teks.
3. Penambahan nilai *enum* baru pada *alert_level* atau *threat_level* di luar nilai yang tercantum pada kontrak.

Perubahan yang tidak kompatibel dapat ditolak *registry* saat pendaftaran. Apabila data yang melanggar kontrak tetap mencapai jalur *ingest*, Aggregator menolak hasil *polling* dan mencatat pelanggaran beserta *correlation ID*. Perubahan tersebut memerlukan penyesuaian kontrak serta pemetaan yang dikoordinasikan antarpihak.

Problem 2 - *Concurrency, Availability, & Graceful Degradation*

Rancangan Solusi

Untuk permasalahan ini, jalur pengambilan data dan jalur permintaan *client* dipisahkan. Permintaan dari *client* tidak pernah memicu pemanggilan ke BMKG maupun PVMBG. Client API meminta data ke Aggregator, dan Aggregator merespons berdasarkan data pada Canonical Store. Pengambilan data dari sumber berjalan secara independen sebagai proses *background*.

Aggregator menjalankan tiga loop polling live secara independen untuk seismic-events, tsunami-warnings, dan volcanic-reports. Tiga loop tambahan mengambil riwayat masing-masing entitas, sedangkan satu loop mencoba kembali pemetaan vulkanik yang tertunda. Pemanggilan jaringan berlangsung di luar transaksi penyimpanan, dan kegagalan ditangani melalui *retry* dengan *exponential backoff* hingga batas yang dikonfigurasi. Setiap loop berjalan pada goroutine dan timer tersendiri dengan interval melalui POLL_INTERVAL, yang wajib diisi dan bernilai 3 detik pada konfigurasi contoh. Pemanggilan jaringan ke PVMBG tidak harus selesai sebelum *loop* BMKG berjalan. Sinkronisasi tetap digunakan untuk pergantian *reader*, *status polling*, dan transaksi *database*. Model ini tidak bersifat bebas lock. Timeout pemanggilan ke BMKG sebesar 2 detik, sedangkan ke PVMBG sebesar 5 detik.

Kondisi outage disimulasikan dengan respons 503 yang dikembalikan secara cepat oleh endpoint data PVMBG. Endpoint `/health` tetap mengembalikan 200 karena proses *mock* masih berjalan. Pilihan ini memisahkan kegagalan penyediaan data dari kegagalan proses dan memungkinkan operator mengaktifkan maupun mematikan outage melalui endpoint admin tanpa restart.

Sesuai laju minimum pada spesifikasi, konfigurasi contoh menghasilkan satu observasi baru setiap 10 detik per `mock` dan dapat dipercepat melalui *environment variable*. Dataset live mempertahankan *seed* dan sebagian observasi terbaru. SQLite persisten menyimpan observasi sumber, sedangkan Aggregator membaca `/history` menggunakan cursor persisten untuk memperoleh observasi yang terlewat dari *polling live*. *Archive* dan *cursor* dipilih agar *recovery* tidak hanya bergantung pada window dataset live.

Untuk gempa dengan *potential_tsunami* bernilai *true*, apabila belum terdapat peringatan tsunami terkait dan pemeriksaan peringatan belum lengkap, observasi disajikan sebagai *notice* berstatus *pending*. HazardEvent dibentuk setelah peringatan terkait ditemukan atau pemeriksaan lengkap memastikan tidak

terdapat peringatan. Apabila dependensi penilaian tidak tersedia, *notice* ditandai dengan status *assessment_unavailable*.

Pada sisi permintaan, Go menangani setiap permintaan pada goroutine tersendiri. Client API menerapkan dua batas jumlah permintaan aktif yaitu, batas global dan batas per identitas *client*. Konfigurasi contoh menetapkan CLIENT_API_MAX_INFLIGHT=128 dan CLIENT_API_MAX_INFLIGHT_PER_CLIENT=32. Kedua variabel wajib disediakan, dan nilai 0 menonaktifkan batas yang bersangkutan. Pembatasan ini mengatur jumlah permintaan yang diproses secara bersamaan, bukan jumlah permintaan per detik.

Urutan pemrosesan permintaan data adalah pembatas global, autentikasi melalui Auth Service, pembatas per client, kemudian handler. Identitas pembatas per client diambil dari `principal.ClientID` hasil verifikasi token. Semua sesi dan token dengan identitas yang sama berbagi kapasitas pada endpoint daftar maupun detail `/hazards` dan `/notices`. Batas tersebut berlaku sama bagi Media, Tim Lapangan, dan BNPB Internal.

Ketika batas global tercapai, permintaan ditolak sebelum pemanggilan ke Auth Service. Ketika batas per client tercapai, permintaan ditolak setelah autentikasi dan sebelum pemanggilan ke Aggregator. Kedua penolakan langsung mengembalikan status 429 dengan header Retry-After: 1, tanpa antrean. *Login*, *refresh*, dan *logout* hanya mengikuti batas global, sedangkan `/health` dan `/ready` dikecualikan dari kedua batas. Setiap pemanggilan dari Client API ke *service* lain memiliki timeout sebesar 5 detik.

Untuk menangani *outage*, *poller* PVMBG mencatat waktu pertama kali *polling* gagal. Selama kondisi tersebut berlangsung, Aggregator tetap menyajikan data vulkanik terakhir yang tersimpan di Canonical Store. Respons menyertakan *source_status* untuk status sumber dan metadata untuk kebaruan serta revisi setiap *record*, di luar objek HazardEvent. Status sumber dapat berupa *fresh*, *stale*, atau *unavailable* berdasarkan hasil *polling* dan batas kebaruan yang dikonfigurasi. Informasi operasional tersebut tidak mengubah attributes. Client API meneruskan *source_status* di luar objek *hazard*, sehingga Media dapat mengetahui status sumber tanpa membaca *attributes*. Setelah *polling* kembali berhasil, status sumber diperbarui menjadi *fresh* dan *stale_since* dikosongkan, sementara *source_status* tetap disertakan dalam respons.

Alasan Pemilihan dan Trade-Off

Alternatif pertama adalah memanggil kedua sumber secara paralel pada setiap permintaan *client*. Pendekatan ini menghilangkan ketergantungan urutan pemanggilan, tetapi permintaan yang

membutuhkan PVMBG tetap menunggu sumber tersebut. *Graceful degradation* masih memerlukan mekanisme tambahan ketika salah satu panggilan gagal, dan beban terhadap sumber meningkat sebanding dengan jumlah client.

Alternatif kedua adalah menempatkan *cache* sebelum pemanggilan ke sumber. Pendekatan ini memperbaiki waktu respons, tetapi permintaan pertama setelah *cache* kedaluwarsa tetap lambat, dan perilaku sistem saat *outage* tetap harus ditangani secara terpisah.

Pendekatan yang dipilih adalah membaca data dari Canonical Store, karena spesifikasi telah menetapkan pola polling dan keberadaan Canonical Store. Jalur baca tidak harus menunggu panggilan ke sumber, dan data terakhir tetap tersedia ketika sumber tidak dapat diakses. Konsekuensinya, data yang dibaca tidak selalu merupakan keadaan sumber pada saat permintaan masuk. Keterlambatan dipengaruhi oleh *interval polling*, waktu respons, pemrosesan, dan jadwal *retry*. Saat terjadi kegagalan, *backoff* dapat menambah keterlambatan. Metadata *source_status* dan kebaruan *record* membantu client menilai kondisi tersebut.

Untuk pembatasan beban, pendekatan *fail-fast* dipilih dibandingkan penggunaan queue. Queue menyebabkan permintaan menumpuk dan pada akhirnya gagal akibat timeout di sisi client, sementara resource tetap terpakai. Respons 429 yang dikembalikan secara langsung memberikan sinyal yang jelas kepada *client* sekaligus melindungi *service* di belakangnya.

Batas global melindungi kapasitas keseluruhan Client API, sedangkan batas per *client* membatasi penggunaan kapasitas operasi baca oleh satu identitas. Penghitung per *client* digunakan bersama oleh seluruh *route* data dan dilindungi *mutex*. *Mutex* hanya dipegang saat pemeriksaan dan perubahan penghitung, kemudian dilepas sebelum handler dijalankan. Slot dilepaskan setelah pemrosesan permintaan berakhir.

Batasan Solusi

1. *Outage* terdeteksi ketika *polling* berikutnya gagal. Dalam kondisi tanpa *backoff* atau penundaan lain, waktu tunggu dipengaruhi oleh *interval polling* dan *timeout* panggilan. Walaupun begitu, jadwal *retry*, waktu pemrosesan, dan sinkronisasi juga dapat memengaruhi waktu deteksi serta pemulihan.
2. Status polling setiap endpoint, waktu keberhasilan terakhir, *stale_since*, jumlah kegagalan, dan jadwal percobaan berikutnya disimpan pada tabel *source_poll_state* di PostgreSQL. Aggregator

menggunakan status tersebut untuk melanjutkan retry dan menyajikan kebaruan data setelah *restart*.

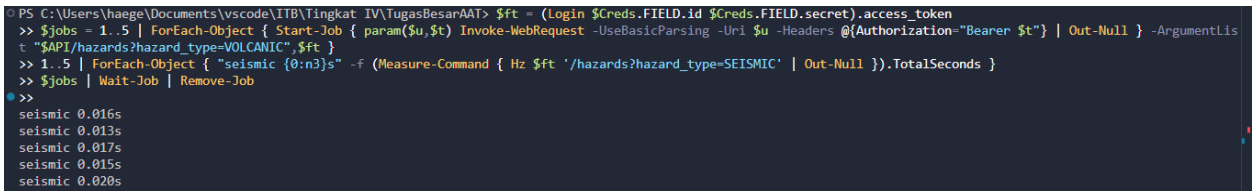
3. Polling live pertama mengambil dataset yang sedang tersedia tanpa parameter since. Polling berikutnya menyertakan since sebesar waktu mulai polling terakhir yang berhasil dikurangi satu detik. Secara terpisah, replay mengambil riwayat secara bertahap melalui `/history` menggunakan cursor persisten. Cursor diperbarui dalam transaksi yang sama dengan penyimpanan hasil observasi dan pekerjaan outbox terkait. Pendekatan ini menghindari pengambilan ulang seluruh dataset. Dataset live mock yang dibatasi menjaga beban polling tetap kecil. Untuk volume sumber yang lebih besar, polling live perlu dilengkapi pagination agar jumlah record per respons tetap terkendali.
4. Setiap permintaan data memerlukan satu kali verifikasi token ke Auth Service. Apabila Auth Service tidak tersedia, permintaan data akan direspons dengan status 503.

Pengujian dan Bukti Penyelesaian



Gambar P2.1. Pengukuran waktu respons langsung endpoint volcanic-reports PVMBG.

Pengujian konkurensi diawali dengan mengatur delay PVMBG menjadi tiga detik sesuai skenario pengujian. Gambar P2.1 memperlihatkan pengukuran langsung terhadap endpoint /volcanic-reports menggunakan kredensial PVMBG, dengan waktu respons 3,06 detik. Pengukuran ini menjadi baseline bahwa sumber vulkanik memang lambat sebelum pengaruhnya terhadap permintaan downstream diperiksa.



Gambar P2.2. Waktu respons seismik saat permintaan vulkanik dijalankan secara paralel.

Selanjutnya, lima permintaan terhadap /hazards?hazard_type=VOLCANIC dijalankan sebagai background job, sementara lima permintaan seismik diukur melalui Client API. Gambar P2.2 menunjukkan waktu respons seismik masing-masing sebesar 25, 10, 17, 12, dan 15 milidetik. Seluruh sampel berada di bawah 300 milidetik meskipun sumber PVMBG memiliki delay sekitar tiga detik. Hasil ini menjadi pemeriksaan awal terhadap pemisahan jalur pembacaan data dari polling sumber; pengukuran p95 menggunakan beban yang lebih besar disajikan pada pengujian berikutnya.

```
MILESTONE 1 LOAD TEST
Mode: mixed | Purpose: isolation | 2 constant VUs | 65s | think time: 50 ms
VUs are concurrent users; capture TCP observations separately for connection evidence.

THROUGHPUT / COUNTS
All HTTP requests (including login and 429): 1792 (29.47/s)
Hazard attempts:                             1788 (29.41/s)
Successful hazard reads (HTTP 200):          1788 (29.41/s)
Login attempts:                              4 (0.07/s)
controlled_rejections_429:                   0 (0.00/s)
unexpected_errors:                          0 (0.00/s)

LATENCY (ms)
Request group                                 p50        p95        p99
All HTTP (includes login / 429)            20.86      38.13      51.69
Login                                      10.98      22.15      23.51
Seismic attempts (includes 429)            20.98      39.29      50.70
Volcanic attempts (includes 429)           20.74      36.57      51.69
Successful hazard reads                    20.87      38.15      51.70
Successful seismic reads                   20.98      39.29      50.70
Successful volcanic reads                  20.74      36.57      51.69

ERROR RATES
http_req_failed (429 stays in denominator): 0.00%
unexpected_failure_rate (excludes 429):     0.00%
hazard_failure_rate (excludes 429/login):   0.00%
login_failure_rate (excludes 429):          0.00%

THRESHOLD RESULTS
PASS hazard_failure_rate: rate<0.01
PASS http_req_duration{kind:seismic}: p(95)<300
PASS http_req_failed: rate<0.01
PASS login_failure_rate: rate<0.01
PASS successful_hazard_duration{kind:seismic}: p(95)<300
PASS successful_hazard_requests: count>0
PASS successful_hazard_requests{kind:seismic}: count>0
PASS successful_hazard_requests{kind:volcanic}: count>0
PASS unexpected_failure_rate: rate<0.01
OVERALL LOAD THRESHOLDS: PASS
Runtime health/restarts and source delay need the separate runbook evidence.

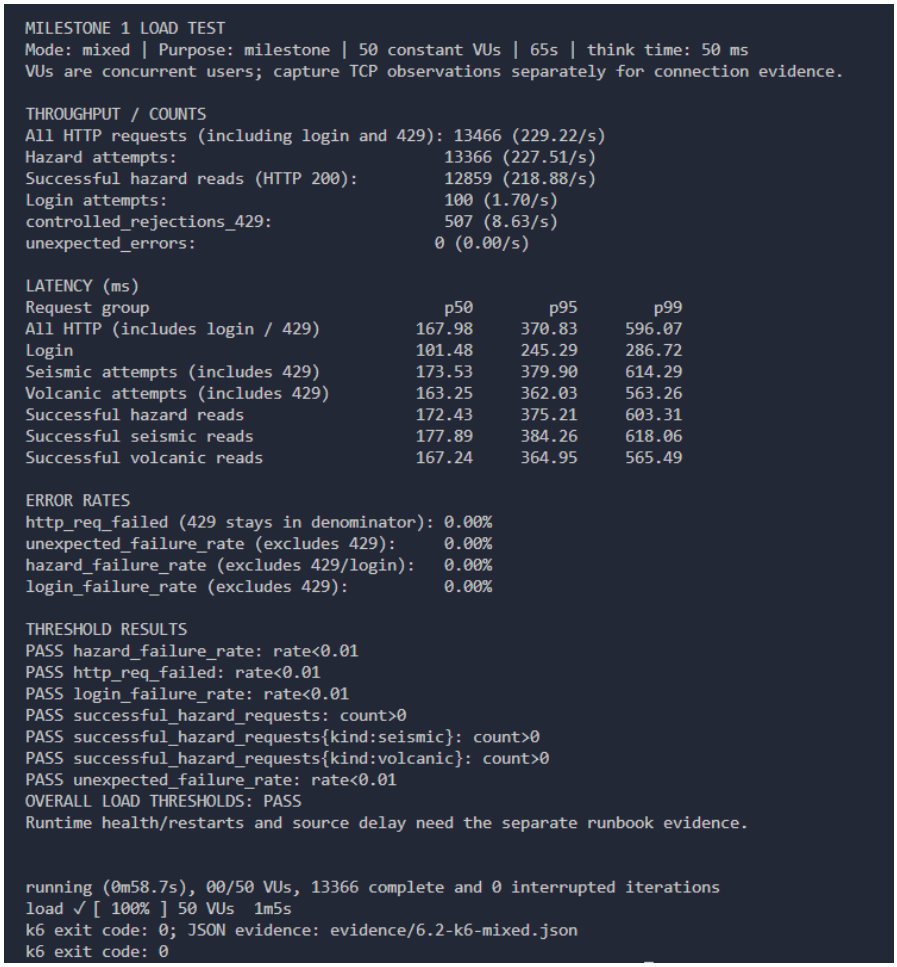

running (1m00.8s), 0/2 VUs, 1788 complete and 0 interrupted iterations
load ✓ [ 100% ] 2 VUs  1m5s
k6 exit code: 0; JSON evidence: evidence/6.1-k6-isolation.json
k6 exit code: 0
```

Gambar P2.3. Uji isolasi latensi dengan dua VUs pada jalur seismik dan vulkanik.

Pengukuran isolasi dilakukan menggunakan k6 dalam mode mixed dengan Purpose=isolation, dua virtual users konstan, durasi konfigurasi 65 detik, dan think time 50 milidetik. Pengujian ini memeriksa apakah permintaan seismik tetap cepat ketika jalur vulkanik turut diakses dan delay sumber PVMBG ditetapkan sebesar tiga detik. Pada Gambar P2.3, respons seismik yang berhasil memiliki p50 20,98 milidetik, p95 39,29 milidetik, dan p99 50,70 milidetik. Nilai p95 berada di bawah batas 300

milidetik. Respons vulkanik yang berhasil memiliki p95 36,57 milidetik, konsisten dengan mekanisme pembacaan Canonical Store yang tidak menunggu panggilan langsung ke PVMBG.

Tercatat 1.792 permintaan HTTP dengan throughput total 29,47 permintaan per detik, termasuk 1.788 pembacaan HazardEvent yang berhasil dengan throughput 29,41 permintaan per detik. Tidak terdapat penolakan HTTP 429 maupun unexpected error. Seluruh threshold dinyatakan PASS dan k6 berakhir dengan exit code 0. Hasil ini mendukung pemenuhan kriteria isolasi latensi P2.1; ketahanan terhadap beban banyak pengguna diuji secara terpisah pada Gambar P2.4.



```
MILESTONE 1 LOAD TEST
Mode: mixed | Purpose: milestone | 50 constant VUs | 65s | think time: 50 ms
VUs are concurrent users; capture TCP observations separately for connection evidence.

THROUGHPUT / COUNTS
All HTTP requests (including login and 429): 13466 (229.22/s)
Hazard attempts:                             13366 (227.51/s)
Successful hazard reads (HTTP 200):          12859 (218.88/s)
Login attempts:                              100 (1.70/s)
controlled_rejections_429:                   507 (8.63/s)
unexpected_errors:                          0 (0.00/s)

LATENCY (ms)
Request group                                  p50        p95        p99
All HTTP (includes login / 429)           167.98     370.83     596.07
Login                                     101.48     245.29     286.72
Seismic attempts (includes 429)           173.53     379.90     614.29
Volcanic attempts (includes 429)          163.25     362.03     563.26
Successful hazard reads                   172.43     375.21     603.31
Successful seismic reads                  177.89     384.26     618.06
Successful volcanic reads                 167.24     364.95     565.49

ERROR RATES
http_req_failed (429 stays in denominator): 0.00%
unexpected_failure_rate (excludes 429):     0.00%
hazard_failure_rate (excludes 429/login):   0.00%
login_failure_rate (excludes 429):          0.00%

THRESHOLD RESULTS
PASS hazard_failure_rate: rate<0.01
PASS http_req_failed: rate<0.01
PASS login_failure_rate: rate<0.01
PASS successful_hazard_requests: count>0
PASS successful_hazard_requests{kind:seismic}: count>0
PASS successful_hazard_requests{kind:volcanic}: count>0
PASS unexpected_failure_rate: rate<0.01
OVERALL LOAD THRESHOLDS: PASS
Runtime health/restarts and source delay need the separate runbook evidence.


running (0m58.7s), 00/50 VUs, 13366 complete and 0 interrupted iterations
load ✓ [ 100% ] 50 VUs  1m5s
k6 exit code: 0; JSON evidence: evidence/6.2-k6-mixed.json
k6 exit code: 0
```

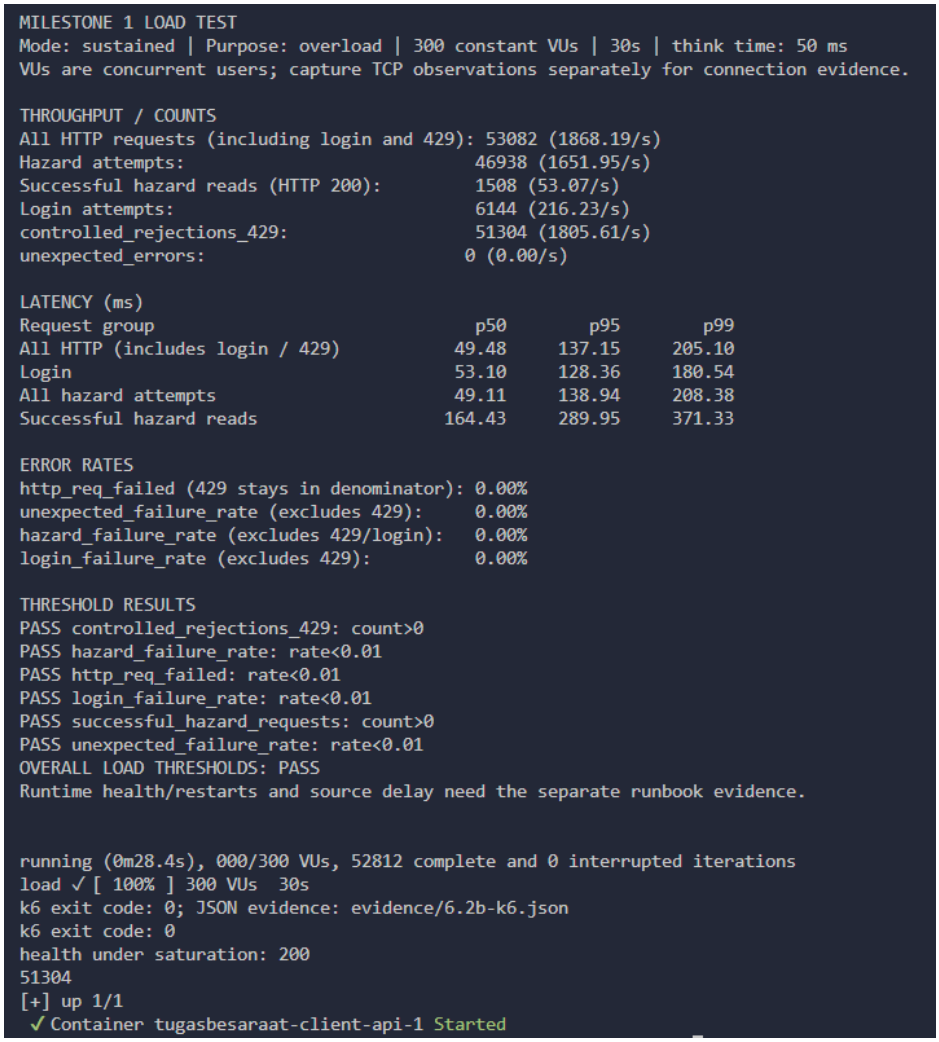
Gambar P2.4. Uji beban berkelanjutan dalam mode mixed dengan 50 VUs dan durasi konfigurasi 65 detik.

Pengujian beban berkelanjutan dilakukan menggunakan mode mixed dengan Purpose=milestone, 50 virtual users konstan, durasi konfigurasi 65 detik, dan think time 50 milidetik. Beban mencakup permintaan seismik dan vulkanik. Pada Gambar P2.4, tercatat 13.466 permintaan HTTP dengan

throughput total 229,22 permintaan per detik. Sebanyak 12.859 pembacaan HazardEvent berhasil, menghasilkan throughput 218,88 permintaan per detik. Latensi seluruh permintaan HTTP memiliki p50 167,98 milidetik, p95 370,83 milidetik, dan p99 596,07 milidetik. Untuk pembacaan HazardEvent yang berhasil, nilainya masing-masing adalah 172,43, 375,21, dan 603,31 milidetik.

Pengujian mencatat 507 penolakan terkontrol HTTP 429 dan nol unexpected error. Unexpected failure rate, hazard failure rate, dan login failure rate di luar HTTP 429 masing-masing bernilai 0,00%, sehingga berada di bawah batas 1%. Seluruh threshold dinyatakan PASS dan exit code k6 bernilai 0. Respons seismik yang berhasil memiliki p95 384,26 milidetik, sedangkan respons vulkanik yang berhasil memiliki p95 364,95 milidetik. Nilai tersebut dilaporkan sebagai karakteristik sistem selama beban berkelanjutan; batas p95 300 milidetik diterapkan pada uji isolasi P2.1, bukan sebagai syarat numerik latensi P2.2.

Jumlah VUs menunjukkan pengguna virtual konkuren. Pembuktian jumlah koneksi paralel serta tidak adanya crash atau restart selama pengujian memerlukan keluaran pengamatan runtime yang terpisah.



Gambar P2.5. Uji overload tambahan dengan 300 VUs selama 30 detik.

Pengujian tambahan dilakukan menggunakan Purpose=overload dengan 300 virtual users dan durasi konfigurasi 30 detik untuk menunjukkan mekanisme penolakan terkontrol. Pada Gambar P2.5, tercatat 53.082 permintaan HTTP dengan throughput total 1.868,19 permintaan per detik. Sebanyak 51.304 permintaan menghasilkan HTTP 429, sementara 1.508 pembacaan HazardEvent berhasil dengan throughput 53,07 permintaan per detik. Unexpected error bernilai nol, dan error rate di luar penolakan terkontrol tercatat 0,00%. Seluruh threshold dinyatakan PASS dengan exit code k6 sebesar 0.

Latensi seluruh permintaan HTTP memiliki p50 49,48 milidetik, p95 137,15 milidetik, dan p99 205,10 milidetik. Pembacaan HazardEvent yang berhasil memiliki p50 164,43 milidetik, p95 289,95 milidetik, dan p99 371,33 milidetik. Perbedaan ini menunjukkan bahwa banyaknya respons penolakan

yang cepat memengaruhi latensi agregat, sehingga latensi pembacaan data yang berhasil perlu dilaporkan secara terpisah.

Pemeriksaan health setelah k6 selesai menghasilkan HTTP 200. Hasil tersebut menunjukkan layanan tersedia setelah pengujian, tetapi tidak menggantikan pemantauan health selama beban berlangsung. Uji overload ini merupakan pengujian tambahan terhadap penolakan terkontrol dan tidak menggantikan uji beban berkelanjutan pada Gambar P2.4.



Gambar P2.6. Respons data selama outage PVMBG dan pemulihan tanpa restart.

Pengujian graceful degradation dilakukan dengan mengaktifkan outage melalui POST /admin/outage. Pada Gambar P2.6, endpoint data PVMBG mengembalikan HTTP 503, sementara endpoint health tetap menghasilkan HTTP 200 karena proses mock masih berjalan. Selama outage,

permintaan data vulkanik melalui Client API tetap memperoleh HTTP 200 dalam 28 milidetik dengan menggunakan data terakhir yang tersimpan. Metadata sumber menunjukkan PVMBG berstatus stale dan menyertakan stale_since, sehingga client dapat mengetahui bahwa data tersebut belum diperbarui. Pada saat yang sama, permintaan seismik tetap menghasilkan HTTP 200 dalam 28 milidetik, dengan status sumber BMKG dan BMKG_TSUNAMI tetap fresh.

Log Aggregator mencatat kegagalan panggilan ke PVMBG beserta correlation ID, status HTTP 503, dan pesan penjadwalan retry. Setelah outage dinonaktifkan, status sumber diperiksa berulang sampai kembali fresh. Runbook menetapkan interval pemeriksaan tiga detik dengan batas tunggu 90 detik untuk mengakomodasi retry backoff dan waktu respons PVMBG. Pada Gambar P2.6, beberapa pemeriksaan masih menghasilkan stale sebelum akhirnya menghasilkan fresh. Permintaan seismik setelah pemulihan tetap memperoleh HTTP 200, dan perbandingan waktu mulai container menghasilkan NO RESTART. Hasil ini menunjukkan pemulihan sumber melalui polling dan retry tanpa restart container.

Model Konkurensi, Penanganan Kegagalan, dan Konfigurasi Load Testing

Model konkurensi yang digunakan terdiri atas satu *goroutine* untuk setiap permintaan pada seluruh *service* HTTP, tiga *loop polling live*, tiga *loop replay* riwayat, dan satu *loop retry* pemetaan vulkanik pada Aggregator, serta pembatas jumlah permintaan aktif secara global dan per identitas *client* pada Client API.

Model tersebut menangani tiga jenis kegagalan yang disebutkan pada spesifikasi sebagai berikut.

1. *Head-of-line blocking* dicegah karena jalur permintaan tidak pernah menunggu respons dari sumber. Permintaan data seismik hanya membaca Canonical Store sehingga lamanya respons PVMBG tidak berpengaruh. Pada sisi *poller*, BMKG dan PVMBG dijalankan pada goroutine yang terpisah sehingga keterlambatan salah satu sumber tidak menghambat sumber lainnya.
2. *Resource exhaustion* ditangani melalui pembatasan jumlah permintaan aktif secara global dan per client. Penolakan global terjadi sebelum pemanggilan ke *Auth Service* maupun Aggregator, sedangkan penolakan per client terjadi setelah verifikasi token dan sebelum pemanggilan ke Aggregator. Permintaan berlebih langsung direspons dengan status 429 tanpa antrean. Seluruh pemanggilan keluar memiliki timeout sehingga permintaan tidak tertahan tanpa batas waktu.
3. Ketiadaan *graceful degradation* diatasi dengan menyajikan data terakhir yang tersedia beserta penanda kemutakhirannya. Ketika PVMBG tidak tersedia, permintaan data vulkanik tetap

direspons dengan status 200 disertai *stale_since*, sedangkan permintaan data seismik tidak terdampak sama sekali.

Load testing dilakukan menggunakan k6 yang dijalankan dari *image* grafana/k6. Skenario pengujian terdapat pada scripts/load-test.js dengan konfigurasi sebagai berikut.

Hal	Nilai
Pola beban	Jumlah *virtual user* konstan selama durasi pengujian
Uji isolasi latensi	Mode mixed, Purpose=isolation, dua VUs konstan dengan durasi konfigurasi 65 detik. Jalur seismik dan vulkanik diakses dengan delay PVMBG ditetapkan tiga detik.
Uji beban berkelanjutan	Mode mixed, Purpose=milestone, 50 VUs konstan dengan durasi konfigurasi 65 detik, mencakup permintaan seismik dan vulkanik.
Uji penolakan terkontrol tambahan	Mode sustained, Purpose=overload, 300 VUs selama 30 detik. Runbook menetapkan batas global sementara sebesar lima dengan batas per client tetap 32.
Identitas yang digunakan	Tim Lapangan. Setiap virtual user melakukan login secara mandiri dan login ulang setelah 75% masa berlaku access token yang diterima melalui expires_in. Pada konfigurasi TTL 60 detik, login ulang dilakukan setelah sekitar 45 detik. Seluruh virtual user menggunakan client_id Tim Lapangan sehingga berbagi batas 32 permintaan aktif, meskipun setiap virtual user memperoleh token berbeda.
Jeda antar permintaan	50 milidetik untuk setiap *virtual user*
Status respons yang diharapkan	200 dan 429. Jumlah respons 429 dihitung dan dilaporkan secara terpisah
Kriteria kelulusan	Uji isolasi: p95 permintaan seismik di bawah 300 milidetik, respons berhasil dari kedua jenis data, dan error rate di bawah 1%. Uji beban: error rate di luar penolakan terkontrol di bawah 1%, respons berhasil dari kedua jenis data, serta pelaporan throughput, latensi, dan jumlah HTTP 429. Bukti koneksi, durasi aktual, dan kestabilan container diperiksa terpisah.
Statistik yang dicatat	Rata-rata, minimum, median (p50), p90, p95, p99, maksimum, *throughput*, jumlah respons 429, dan error selain penolakan terkontrol.
Pemantauan *runtime*	Pengamatan koneksi TCP, health, container ID, waktu mulai, dan restart count dilakukan bersamaan dengan uji beban. Jumlah VUs tidak dianggap sebagai bukti langsung jumlah koneksi TCP.

Problem 3 - *Service-to-Service Authentication & Trust*

Rancangan Solusi

Kredensial dipisahkan ke dalam tiga *trust zone* yang tidak saling berbagi nilai kredensial.

1. *Trust zone* pertama adalah akses ke BMKG. *Mock* BMKG memverifikasi kunci yang dikirimkan melalui *header* X-BMKG-Key.
2. *Trust zone* kedua adalah akses ke PVMBG, yang menggunakan *header* Authorization berisi *bearer token*.
3. *Trust zone* ketiga adalah *downstream client*.

Kedua mock mengembalikan 401 ketika kredensial tidak disertakan dan 403 ketika nilainya tidak valid. Di antara komponen BNPB yang memanggil sumber, hanya Aggregator yang menerima kunci BMKG dan token data PVMBG. Mock masing-masing juga menyimpan kredensial pembanding melalui konfigurasi miliknya. Client API, Auth Service, *consumer*, dan *client downstream* tidak memerlukan kedua kredensial sumber tersebut.

Endpoint `/admin/*` PVMBG dilindungi PVMBG_ADMIN_TOKEN yang berbeda dari token akses data, sehingga pemegang token data tidak dapat memicu *outage* atau mengganti *writer schema*. Endpoint pergantian *reader* pada Aggregator menggunakan AGGREGATOR_ADMIN_TOKEN tersendiri dan tidak dipublikasikan ke *host*. Token ini dibaca dari konfigurasi Aggregator dan harus berbeda dari kedua kredensial sumber. Perintah demo dijalankan melalui container Aggregator agar tidak perlu menambahkan port publik untuk kontrol reader.

Pada lingkup *downstream client*, tiga identitas beserta *client secret* dan perannya ditetapkan melalui konfigurasi Auth Service sebelum *startup.* Pendaftaran client baru saat sistem berjalan tidak diimplementasikan. Saat login, Auth Service memverifikasi client ID dan *secret*, lalu menerbitkan access token serta refresh token. Masa berlaku dibaca dari ACCESS_TOKEN_TTL_SEC dan REFRESH_TOKEN_TTL_SEC, konfigurasi contoh menggunakan 60 detik dan 24 jam. ACCESS_TOKEN_TTL_SEC wajib disediakan dan diterapkan seragam untuk ketiga identitas. Demonstrasi kedaluwarsa alami menggunakan Tim Lapangan sesuai spesifikasi. Peran ditentukan server berdasarkan identitas, bukan berdasarkan flag yang dikirim client.

Token berupa nilai acak sepanjang 32 byte (opaque token) yang tidak memuat identitas maupun role. Auth Service menyimpan hash SHA-256 token beserta metadata sesi pada SQLite miliknya. Pada

setiap permintaan data, Client API memverifikasi token ke Auth Service. Endpoint internal Auth Service dilindungi AUTH_INTERNAL_KEY yang digunakan oleh Auth Service dan Client API. Consumer tidak memanggil Auth Service.

Penegakan hak akses dilakukan pada Client API. Media hanya menerima tujuh field dengan klasifikasi Ringkasan. Respons untuk Media dibentuk dari struktur data tersendiri yang memang tidak memiliki koordinat, *source_ref_id*, maupun *attributes*, sehingga field mentah tidak mungkin ikut terkirim. Apabila Media secara eksplisit meminta data mentah melalui parameter *view=raw*, permintaan tersebut direspons dengan status 403.

Ketika *access token* kedaluwarsa, client menukarkan *refresh token* untuk memperoleh pasangan token baru. Proses penukaran ini mengganti *hash access token* dan *hash refresh token* pada sesi yang sama dalam satu operasi atomik. Akibatnya, *access token* dan *refresh token* lama langsung tidak berlaku. Client TUI melakukan penukaran ini secara otomatis ketika menerima respons 401, kemudian mengulang permintaan tanpa mengharuskan pengguna melakukan login ulang.

Sebagai pengamanan tambahan, Aggregator menolak startup jika kunci BMKG sama dengan token PVMBG, atau token admin Aggregator sama dengan salah satu kredensial sumber. Mock PVMBG menolak startup jika token data sama dengan token admin. Auth Service menolak client ID atau client secret yang duplikat.

Alasan Pemilihan dan Trade-Off

Pertimbangan utama pada bagian ini terletak pada bentuk token. Rancangan awal menggunakan JWT yang ditandatangani dengan HS256. Keunggulan JWT adalah kemampuannya untuk diverifikasi tanpa perlu menghubungi Auth Service. Namun, spesifikasi mewajibkan agar token lama segera ditolak setelah proses *refresh*. JWT yang telah diterbitkan tetap sah hingga masa berlakunya habis, sehingga penolakan lebih awal tetap memerlukan *revocation list* di sisi *server*. Dengan adanya kebutuhan tersebut, keunggulan utama JWT menjadi tidak relevan.

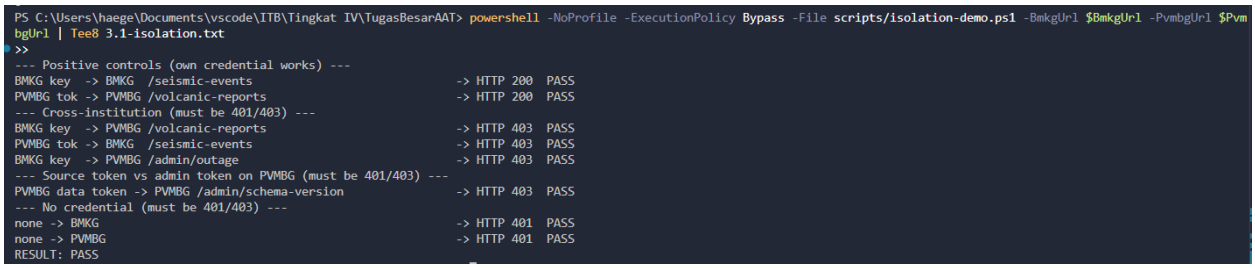
Oleh karena itu, rancangan diubah menggunakan *opaque token* yang diverifikasi melalui Auth Service. Pencabutan token menjadi sederhana karena cukup dilakukan dengan mengganti nilai *hash* di basis data. Tidak terdapat *signing key* yang berisiko bocor, dan token tidak membawa informasi yang dapat dibaca oleh pihak lain. Kompensasinya adalah satu pemanggilan tambahan ke Auth Service pada setiap permintaan data, serta Auth Service menjadi dependensi bagi seluruh operasi baca.

Opsi penggabungan Auth Service ke dalam Client API juga dipertimbangkan. Kedua komponen tersebut dipisahkan agar *client secret* hanya disimpan oleh satu *service* yang tidak terekspos ke luar, serta agar perubahan pada mekanisme autentikasi dapat di-*deploy* tanpa memengaruhi Client API.

Batasan Solusi

1. *Endpoint* internal *Auth Service* dilindungi oleh satu kunci statis yang digunakan bersama oleh *Auth Service* dan Client API. Autentikasi dengan identitas unik untuk setiap *service* maupun mTLS belum diterapkan.
2. Komunikasi antar-*container* tidak dienkripsi. Jaringan internal Docker Compose dianggap sebagai zona tepercaya untuk lingkup M1.
3. Tidak terdapat pembatasan jumlah percobaan *login* per *client*. Pembatasan yang tersedia hanya berupa batas jumlah permintaan aktif secara keseluruhan.
4. Token untuk BMKG dan PVMBG bersifat statis dan tidak dirotasi.

Pengujian dan Bukti Penyelesaian



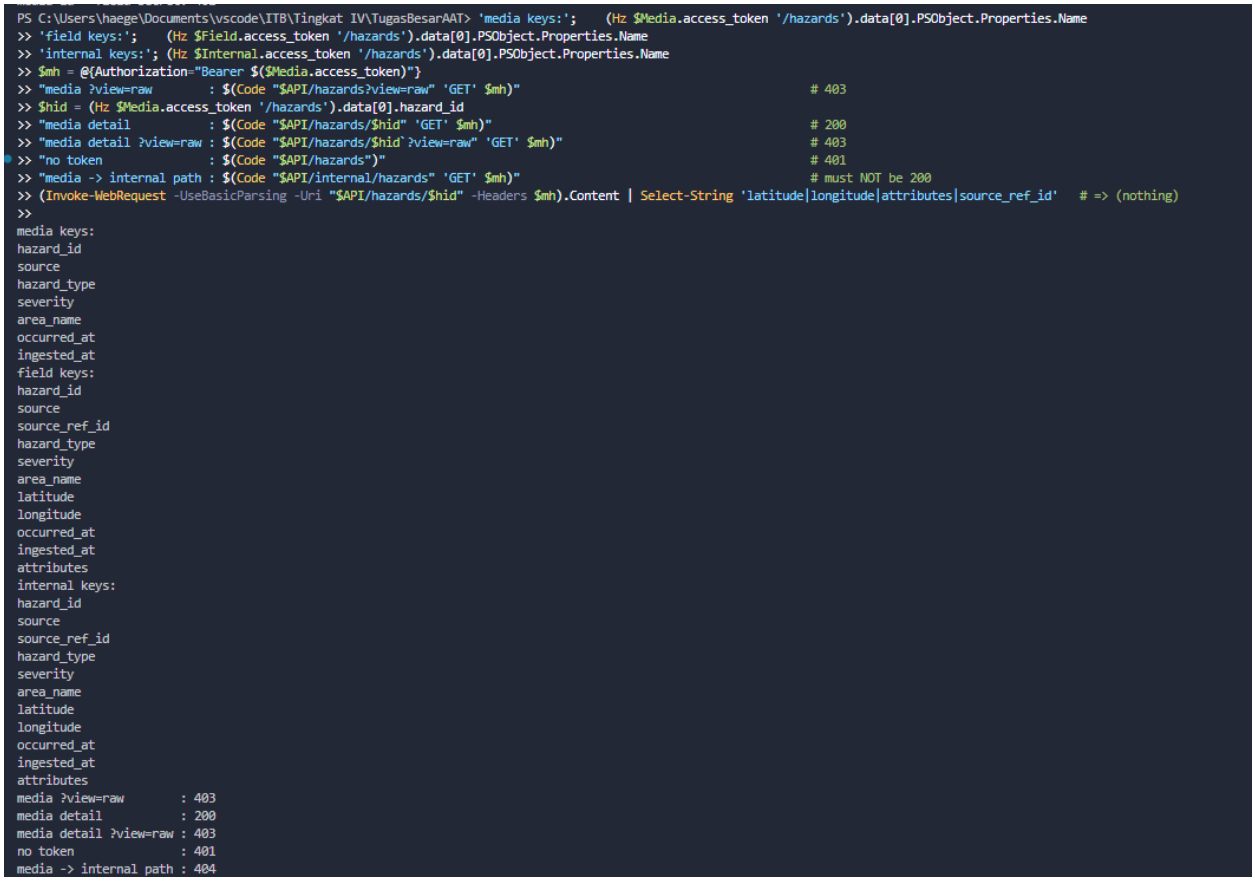
Gambar P3.1. Hasil kontrol positif dan penolakan kredensial yang tidak sesuai.

Pengujian pemisahan domain kepercayaan dilakukan dengan kontrol positif dan percobaan penggunaan kredensial lintas instansi. Hasil pada Gambar P3.1 menunjukkan HTTP 200 untuk kredensial yang benar, HTTP 403 untuk penggunaan kredensial lintas instansi, dan HTTP 401 untuk permintaan tanpa kredensial. Token akses data PVMBG juga ditolak ketika digunakan pada endpoint administrasi. Hasil tersebut menunjukkan bahwa kredensial sumber dan kredensial administrasi memiliki batas penggunaan yang berbeda.



Gambar P3.2. Login tiga identitas client dan penolakan pasangan kredensial silang.

Pemisahan identitas downstream diperiksa melalui login Media, Tim Lapangan, dan BNPB Internal. Gambar P3.2 menunjukkan ketiga identitas memperoleh peran masing-masing, sedangkan kombinasi client ID Media dengan secret Tim Lapangan menghasilkan HTTP 401. Pengujian hak akses pada Gambar P3.3 kemudian memperlihatkan bahwa respons Media hanya memuat field ringkasan. Field source_ref_id, latitude, longitude, dan attributes tersedia bagi Tim Lapangan serta BNPB Internal, tetapi tidak muncul pada keluaran Media. Permintaan Media terhadap view=raw, baik pada daftar maupun detail HazardEvent, ditolak dengan HTTP 403. Pembatasan ini ditegakkan oleh server, sebagaimana terlihat dari status respons dan field yang dikembalikan.

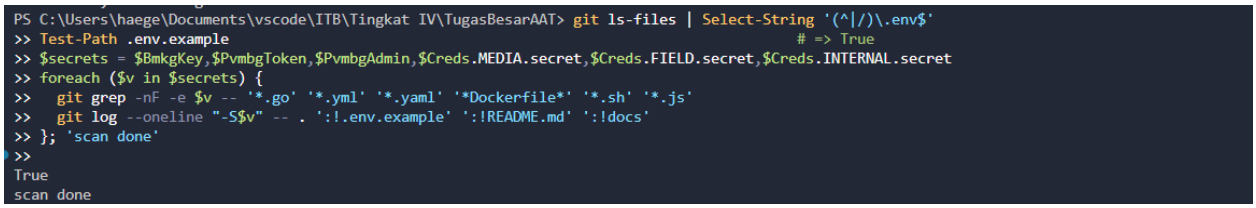


Gambar P3.3. Field respons per client dan penolakan akses Media terhadap data mentah.



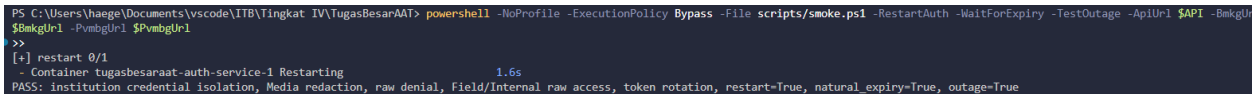
Gambar P3.4. Kedaluwarsa access token, refresh, dan penolakan token lama.

Siklus token Tim Lapangan diuji dengan menunggu selama expires_in ditambah satu detik. Pada Gambar P3.4, access token diterima sebelum kedaluwarsa dan ditolak dengan HTTP 401 setelah masa berlakunya berakhir. Refresh menggunakan refresh token menghasilkan access token baru yang diterima dengan HTTP 200, sementara access token lama dan penggunaan ulang refresh token lama ditolak. Pada langkah refresh, pembaruan token dilakukan menggunakan refresh token dari sesi yang sudah tersedia, tanpa memanggil endpoint login kembali. Hasil ini menunjukkan bahwa sesi dapat dilanjutkan setelah access token kedaluwarsa melalui alur refresh, sedangkan token lama tidak dapat digunakan kembali.



Gambar P3.5. Pemeriksaan .env.example dan pencarian nilai secret pada repositori.

Pemeriksaan konfigurasi pada Gambar P3.5 menunjukkan bahwa berkas .env.example tersedia, sementara pemeriksaan berkas .env yang terlacak Git dan pencarian nilai secret yang diuji tidak menghasilkan temuan pada keluaran yang ditampilkan. Pemeriksaan mencakup berkas kode tertentu serta pencarian nilai secret tersebut pada riwayat Git. Hasil ini mendukung pemisahan kredensial dari repositori, dengan batasan bahwa pencarian hanya mendeteksi nilai secret dan cakupan berkas yang diperiksa.



Gambar P3.6. Ringkasan smoke test autentikasi dengan restart Auth Service dan kedaluwarsa alami.

Pengujian autentikasi dilengkapi dengan smoke test menggunakan opsi RestartAuth dan WaitForExpiry. Gambar P3.6 memperlihatkan Auth Service dijalankan ulang dan script menghasilkan status PASS untuk pemisahan kredensial instansi, pembatasan field Media, penolakan akses mentah, akses mentah Tim Lapangan dan BNPB Internal, serta rotasi token. Ringkasan juga mencatat restart=True dan natural expiry=True. Hasil ini menjadi bukti tambahan terhadap skenario autentikasi yang diuji, sedangkan urutan respons sebelum kedaluwarsa, setelah kedaluwarsa, dan setelah refresh tetap ditunjukkan secara rinci pada Gambar P3.4.

Mekanisme Autentikasi dan Analisis Keamanan

Mekanisme yang digunakan terdiri atas kredensial terpisah untuk setiap instansi sumber dengan format *header* yang berbeda, *opaque token* berumur pendek untuk *downstream client* yang diverifikasi ke Auth Service pada setiap permintaan, rotasi *refresh token*, serta pembatasan *field* di sisi *server* berdasarkan peran (*role-based*).

Mekanisme tersebut dinilai tidak menimbulkan celah keamanan baru, dengan alasan sebagai berikut.

1. Kebocoran kredensial pada satu *trust zone* tidak dapat dimanfaatkan pada *trust zone* lainnya. *Downstream client* tidak pernah memiliki kunci BMKG maupun token PVMBG, dan kredensial milik satu instansi akan ditolak oleh instansi lainnya.
2. Token *client* sulit ditebak karena dibangkitkan dari 32 byte acak menggunakan crypto/rand. Tidak terdapat *signing key* yang apabila bocor, memungkinkan pihak lain membuat token sendiri.
3. Basis data Auth Service hanya menyimpan nilai *hash*. Pihak yang berhasil membaca basis data tersebut tidak akan memperoleh token yang dapat digunakan.
4. Setiap proses *refresh* menghasilkan pasangan *access* token dan *refresh* token baru. Kedua token lama langsung tidak berlaku setelah proses tersebut berhasil.
5. Auth Service tidak terekspos ke *host* dan tidak berada dalam jaringan yang sama dengan *client*. *Endpoint* internalnya menolak setiap permintaan yang tidak menyertakan kunci internal.
6. Pembatasan *field* tidak bergantung pada kepatuhan *client*. *Field* mentah tidak pernah dikirimkan kepada Media, sehingga tidak ada data yang dapat diakses dengan memodifikasi aplikasi *client*.

7. Pesan kesalahan pada proses *login* dan verifikasi token dibuat seragam, sehingga tidak mengungkapkan apakah *client ID* yang digunakan terdaftar atau tidak.

Problem 4 - Arsitektur Microservice dan *Flexible Storage*

Rancangan Solusi

Sistem dipecah menjadi beberapa *service* yang masing-masing memiliki Dockerfile, *container*, dan penyimpanan tersendiri.

Service	Tanggung jawab	Penyimpanan yang dimiliki
Mock BMKG	Menyajikan data gempa bumi dan peringatan tsunami	Dataset live di memori dan SQLite persisten pada volume tersendiri
Mock PVMBG	Menyajikan laporan gunung api, *toggle* skema, dan *toggle outage*	Dataset live di memori dan SQLite persisten pada volume tersendiri
Aggregator	*Polling*, pemetaan, deduplikasi, penerbitan *event*, dan API baca internal	Canonical Store (PostgreSQL)
Auth Service	*Login*, penerbitan dan rotasi token, serta verifikasi token	SQLite berisi data sesi
Client API	Titik masuk *client*, penegakan hak akses, dan pembatasan beban	Tidak ada
Broker	Mendistribusikan *event* kepada *consumer*	*Stream* JetStream pada *volume*
Schema Registry	Menyimpan dan memvalidasi skema	PostgreSQL tersendiri
Consumer dashboard	Menyimpan salinan lengkap setiap *event*	SQLite tersendiri
Consumer notifier	Mencatat ringkasan notifikasi untuk setiap *event*	SQLite tersendiri

Tidak terdapat dua *service* yang mengakses penyimpanan yang sama. Kepemilikan Canonical Store ditegakkan melalui isolasi jaringan. PostgreSQL hanya terhubung ke jaringan *store*, dan satu-satunya *service* lain pada jaringan tersebut adalah Aggregator. PostgreSQL juga tidak mengekspos *port* ke *host*. Client API memperoleh data melalui *endpoint* `/internal/hazards` milik Aggregator.

Agar setiap *service* dapat dihentikan dan dijalankan secara independen, ketergantungan urutan *startup* dihindari. Aggregator tetap berjalan dan merespons *health check* meskipun PostgreSQL atau *broker* belum siap, kemudian membangun koneksi secara otomatis setelah keduanya tersedia. Kedua *consumer* melakukan *reconnect* ke *broker* secara berkala. Client API mengembalikan status 503 apabila *service* di belakangnya tidak tersedia, kemudian pulih tanpa memerlukan *restart*. Tidak terdapat kode yang digunakan bersama antar-*service*. Tipe data yang sama didefinisikan ulang pada setiap *service* agar masing-masing dapat di-*build* secara independen.

Pada sisi penyimpanan, tabel *hazard_events* memiliki kolom reguler untuk field baku yang sering digunakan dalam penyaringan serta kolom *attributes* bertipe JSONB untuk atribut spesifik sumber. Setelah reader yang menerima *confidence_level* aktif dan observasi yang memuatnya diproses, nilai tersebut tersimpan di attributes tanpa ALTER TABLE atau migrasi khusus field baru. Record lama dan baru dapat berada pada tabel yang sama serta dibaca melalui API yang sama. Revision dan metadata kebaruan disimpan atau disajikan terpisah dari objek *HazardEvent*. Keduanya tidak menambah field baku pada kontrak kanonik.

SQLite pada *mock* dipilih agar setiap sumber tetap memiliki observasinya sendiri tanpa database server tambahan. Consumer juga menggunakan SQLite untuk proyeksi dan identitas pesan yang diproses. Penyimpanan *consumer* tidak digunakan sebagai Canonical Store dan tidak menjadi jalur baca Client API. Setiap volume dipasang pada service pemiliknya, sementara akses antarkomponen dilakukan melalui HTTP atau broker.

Alasan Pemilihan dan Trade-Off

Batas setiap *service* ditentukan berdasarkan tiga pertimbangan, yaitu kepemilikan data, frekuensi perubahan komponen, dan dampak yang ditimbulkan apabila komponen tersebut gagal. Aggregator menangani polling, pemetaan, dan API baca internal karena ketiganya berada dalam kepemilikan data kanonik yang sama. *Service* baca terpisah tetap harus meminta data melalui Aggregator untuk mematuhi kepemilikan Canonical Store. Pemisahan tersebut dapat berguna untuk penskalaan atau pengelolaan API secara independen, tetapi menambah *network hop* dan komponen operasional. Untuk lingkup M1, Client

API sudah menangani antarmuka *downstream* dan penegakan hak akses, sehingga API baca internal dipertahankan pada Aggregator.

Auth Service dipisahkan dari Client API karena keduanya berubah atas alasan yang berbeda dan menyimpan data yang berbeda. Perbaikan pada mekanisme autentikasi semestinya tidak mengharuskan komponen penyajian data turut di-*deploy* ulang, dan hal tersebut merupakan permasalahan yang diangkat pada skenario yang terlampir pada dokumen spesifikasi.

Untuk penyimpanan, dilakukan perbandingan terhadap tiga alternatif.

	PostgreSQL dengan JSONB	**MongoDB**	**Redis**
Atribut dinamis tanpa migrasi	Ya, melalui kolom JSONB	Ya, didukung secara *default*	Ya
Transaksi untuk menyimpan *event* dan antrean penerbitan secara bersamaan	Ya, didukung secara *default*	Hanya apabila dijalankan sebagai *replica set*	Terbatas
Deduplikasi melalui *unique index*	Ya	Ya	Harus diimplementasikan sendiri
Penyaringan berdasarkan beberapa kolom	Ya, dengan indeks reguler	Ya	Indeks harus diimplementasikan sendiri
Tabel referensi gunung api	Sesuai	Kurang sesuai	Tidak sesuai
Kemudahan penyimpanan dokumen JSON	Memerlukan pemetaan ke kolom	Paling ringkas	Ringkas

MongoDB sesuai untuk dokumen dengan atribut dinamis. Transaksi multi-document tersedia pada replica set dan sharded cluster, sedangkan deployment standalone tidak mendukungnya. Alasan memilih PostgreSQL adalah tersedianya transaksi untuk hazard dan outbox, unique index, serta query referensi dalam konfigurasi database yang digunakan proyek, tanpa menambahkan konfigurasi replica set.

HazardEvent juga sebagian besar memiliki field baku yang sering difilter, sehingga kombinasi kolom reguler dan JSONB sesuai dengan pola aksesnya.

Konsekuensinya adalah kode penyimpanan yang perlu memetakan field baku ke kolom. JSONB memvalidasi bentuk JSON, tetapi database tidak menerapkan schema per field untuk isi attributes. Validasi tipe, nilai yang diizinkan, dan aturan tambahan pada data sumber dilakukan Aggregator sebelum penyimpanan. Penambahan atribut tidak memerlukan perubahan tabel, sementara perubahan pada field baku tetap memerlukan penyesuaian model dan storage.

Batasan Solusi

1. Fleksibilitas skema hanya berlaku untuk atribut tambahan dari sumber. Perubahan pada daftar *field* baku HazardEvent tetap memerlukan perubahan struktur tabel.
2. Belum terdapat indeks pada isi attributes. Pencarian berdasarkan nilai atribut akan lambat apabila volume data besar.
3. Skema dasar dibuat melalui init.sql, sedangkan Aggregator menerapkan migrasi lifecycle yang idempoten pada skema yang sudah ada. Belum tersedia *migration tool* untuk perubahan tabel.
4. Kode *decoder* Avro dan tipe data HazardEvent terduplikasi pada beberapa *service*. Hal ini merupakan keputusan yang disengaja untuk menghindari *shared library*, dengan risiko terjadinya perbedaan antarsalinan apabila tidak dipelihara dengan baik.
5. PostgreSQL dijalankan sebagai satu *instance* tanpa replikasi.

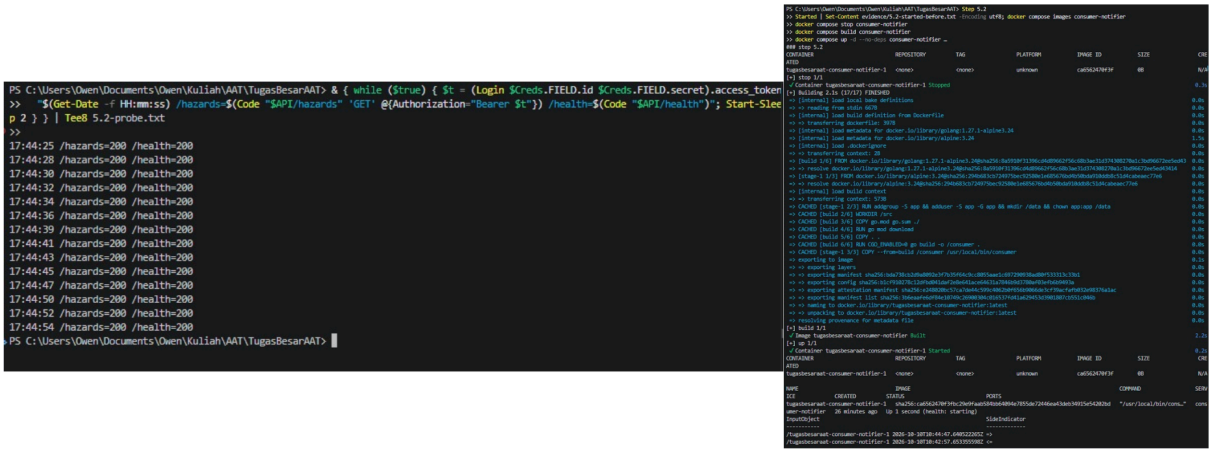
Pengujian dan Bukti Penyelesaian

```
PS C:\Users\Owen\Documents\Owen\Kuliah\AAT\TugasBesarAAT> docker compose ps --format 'table {{.Service}}\t{{.Image}}\t{{.Status}}\t{{.Ports
}}' | Tee8 5.1-containers.txt
>>
SERVICE             IMAGE                                STATUS                  PORTS
aggregator          tugasbesaraat-aggregator             Up 5 minutes (healthy)
auth-service        tugasbesaraat-auth-service           Up 5 minutes (healthy)
bmkg-mock           tugasbesaraat-bmkg-mock              Up 5 minutes (healthy)   0.0.0.0:18081->8081/tcp, [::]:18081->8081/tcp
broker              tugasbesaraat-broker                 Up 5 minutes (healthy)   4222/tcp, 6222/tcp, 8222/tcp
client-api          tugasbesaraat-client-api             Up 5 minutes (healthy)   0.0.0.0:18080->8080/tcp, [::]:18080->8080/tcp
consumer-dashboard  tugasbesaraat-consumer-dashboard     Up 5 minutes (healthy)
consumer-notifier   tugasbesaraat-consumer-notifier      Up 5 minutes (healthy)
postgres            postgres:16-alpine3.24               Up 5 minutes (healthy)   5432/tcp
pvmbg-mock          tugasbesaraat-pvmbg-mock             Up 5 minutes (healthy)   0.0.0.0:18082->8082/tcp, [::]:18082->8082/tcp
registry-db         postgres:16-alpine3.24               Up 5 minutes (healthy)   5432/tcp
schema-registry     tugasbesaraat-schema-registry        Up 5 minutes (healthy)   8080/tcp, 8443/tcp, 9000/tcp
```

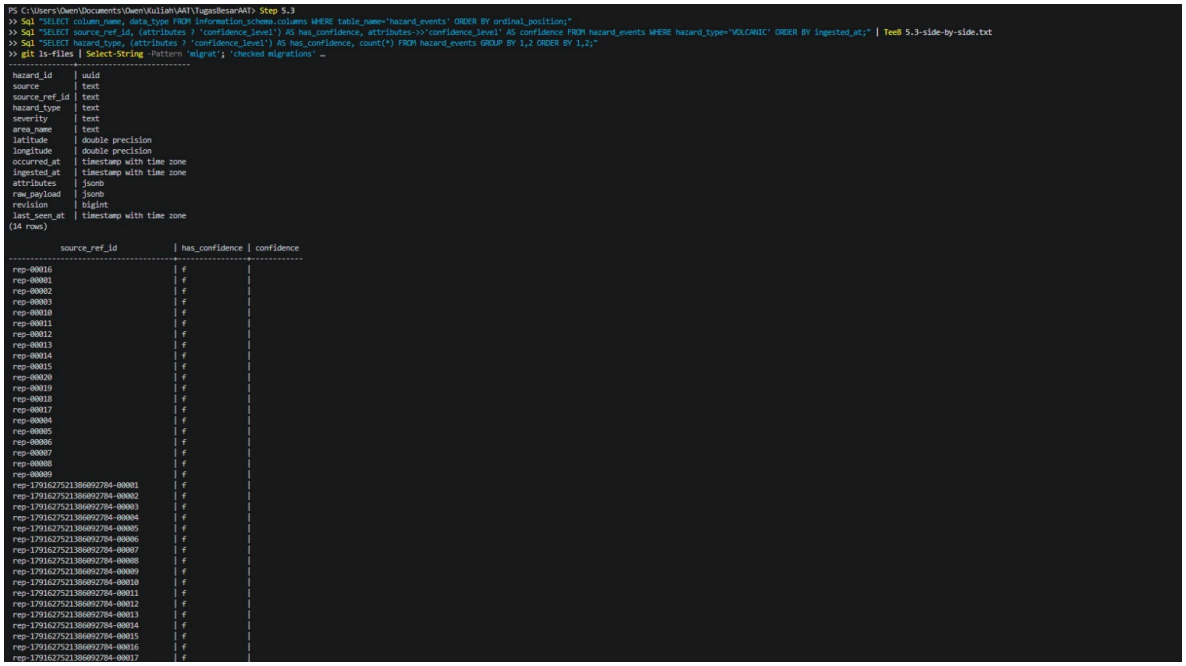
Gambar P4.1. Daftar service, image, status, dan port container.

Pengujian independensi deployment diawali dengan memeriksa daftar service, image, status, dan port melalui Docker Compose. Gambar P4.1 memperlihatkan komponen sistem berjalan sebagai container

terpisah. Selanjutnya, consumer notifier dihentikan, di-build ulang, dan dijalankan kembali sebagaimana ditunjukkan pada Gambar P4.2. Output terminal memperlihatkan build selesai dan container notifier kembali berstatus Started. Perbandingan waktu mulai container hanya menampilkan perubahan pada consumer notifier. Sementara itu, probe berkala pada terminal kanan tetap memperoleh HTTP 200 dari /hazards dan /health selama proses tersebut. Hasil ini menunjukkan bahwa rebuild notifier dapat dilakukan tanpa menjalankan ulang container lain maupun menghentikan jalur pembacaan data yang tidak bergantung padanya.



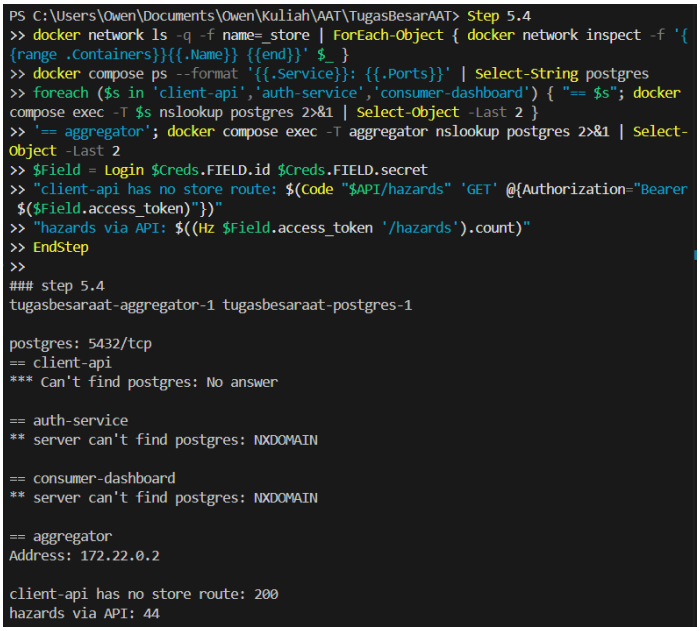
Gambar P4.2. Rebuild dan pengaktifan ulang notifier dengan probe API tetap menghasilkan HTTP 200.



Gambar P4.3. Struktur JSONB dan pembacaan record dengan variasi confidence_level.

Fleksibilitas penyimpanan diperiksa melalui struktur tabel dan isi kolom attributes. Gambar P4.3 memperlihatkan penggunaan tipe JSONB serta record vulkanik dengan dan tanpa confidence_level yang dapat dibaca dalam tabel yang sama. Rekapitulasi menghasilkan 126 record VOLCANIC tanpa field tersebut dan 2 record dengan field tersebut. Hasil ini menunjukkan bahwa variasi atribut dapat disimpan berdampingan tanpa menjadikan confidence_level sebagai kolom tersendiri. Pemeriksaan struktur tabel menunjukkan bahwa confidence_level tidak ditambahkan sebagai kolom tersendiri, melainkan dibaca dari

attributes. Keluaran riwayat perubahan yang ditampilkan menjadi bukti pendukung, tetapi bukan perbandingan langsung struktur database sebelum dan sesudah perubahan skema.



Gambar P4.4. Isolasi jaringan PostgreSQL dan resolusi nama dari service yang diuji.

Isolasi Canonical Store diperiksa melalui keanggotaan jaringan Docker, publikasi port, dan resolusi nama service. Pada Gambar P4.4, jaringan PostgreSQL hanya mencakup PostgreSQL dan Aggregator, serta port database tidak dipublikasikan ke host. Client API, Auth Service, dan consumer dashboard yang diuji tidak dapat me-resolve nama postgres, sedangkan Aggregator berhasil. Bukti jaringan ini, bersama log panggilan Client API menuju /internal/hazards pada P1, mendukung mekanisme pembacaan data melalui API Aggregator.

Dasar Pembagian Service dan Perbandingan Penyimpanan

Batas *service* ditentukan berdasarkan kepemilikan data dan alasan perubahan. *Mock* dipisahkan per *instance* karena merepresentasikan pihak yang otonom. Aggregator memiliki Canonical Store dan menyediakan API bagi komponen yang memerlukan datanya. Auth Service memiliki sesi serta konfigurasi identitas dan dapat di-*deploy* terpisah dari penyajian data. Client API menjadi antarmuka downstream untuk autentikasi dan pembacaan data BNPB. *Mock* memiliki port host tersendiri untuk demonstrasi, sedangkan Grafana menyediakan antarmuka *monitoring*. Setiap *consumer* berdiri sendiri agar kegagalannya terisolasi. Pemisahan API baca tambahan dan penggabungan Auth Service dengan

Client API dipertimbangkan berdasarkan biaya komunikasi, kepemilikan data, dan kebutuhan deployment yang dijelaskan sebelumnya.

Perbandingan PostgreSQL dengan JSONB, MongoDB, dan Redis mempertimbangkan atribut dinamis, transaksi outbox, deduplikasi, serta query referensi. PostgreSQL dipilih karena memenuhi kebutuhan tersebut melalui transaksi dan indeks SQL sekaligus menyediakan JSONB. Validasi isi attributes tetap menjadi tanggung jawab Aggregator. Konsekuensinya adalah pemetaan field baku ke kolom dan pengelolaan perubahan tabel untuk perubahan kontrak kanonik, sementara atribut sumber tambahan dapat disimpan tanpa migrasi.

Problem 5 - Fan-out Event Distribution/Pub-sub

Rancangan Solusi

Aggregator tidak pernah memanggil *consumer* secara langsung. Aggregator menerbitkan *event* kanonik melalui `hazard.events` dan event *lifecycle* melalui `report.lifecycle`, dan setiap pihak yang berlangganan akan menerima *event* tersebut.

Agar *event* tidak hilang ketika *broker* tidak tersedia, digunakan *transactional outbox pattern*. Ketika Aggregator menyimpan HazardEvent baru, Aggregator juga menulis satu baris ke tabel *hazard_outbox* dalam transaksi yang sama. Sebuah proses *publisher* di dalam Aggregator mengambil baris tertua, menerbitkannya ke JetStream, kemudian menghapus baris tersebut hanya setelah JetStream mengonfirmasi penerimaan. Apabila *broker* tidak tersedia, baris tersebut tetap tersimpan dalam antrean dan proses *polling* tetap berjalan.

Broker yang digunakan adalah NATS JetStream. *Stream* bernama HAZARDS menampung *subject* hazard.events, disimpan pada *file storage*, dan mempertahankan pesan sesuai dengan konfigurasi retensinya. Pesan dikodekan dalam format Avro dan dibungkus dalam *envelope* yang memuat versi *envelope*, waktu penerbitan, serta *correlation ID* dari siklus *polling* yang menghasilkan pesan tersebut.

Setiap *consumer* berlangganan sebagai *durable consumer* dengan nama tersendiri. *Broker* menyimpan posisi baca setiap *durable consumer* secara terpisah. *Consumer* mengambil satu pesan, melakukan *decoding*, menyimpan hasilnya ke SQLite, dan baru setelah itu mengirimkan *acknowledgement* (*ack*). Apabila *ack* tidak diterima, *broker* akan mengirimkan ulang pesan tersebut.

Pada subject hazard.events, hanya *HazardEvent* baru yang diterbitkan. Penerimaan ulang sumber dan *source_ref_id* yang sama mempertahankan *hazard_id*. Jika isi kanonik berubah secara material, revision meningkat dan event *hazard_corrected* diterbitkan melalui *report.lifecycle* dengan snapshot terbaru. Penerimaan ulang tanpa perubahan isi tidak menghasilkan koreksi baru. Revision merupakan metadata pengelolaan versi, bukan penambahan field pada objek *HazardEvent.* Consumer menerapkan koreksi ketika revision lebih tinggi daripada versi lokal agar pesan lama atau duplikat tidak menimpa data terbaru.

Consumer ketiga dapat ditambahkan melalui *profile* Docker Compose. *Consumer* tersebut dibangun dari kode dan Dockerfile yang sama dengan *consumer* dashboard, dengan nama *durable*

consumer dan volume tersendiri. Penambahan ini tidak memerlukan perubahan maupun *restart* pada Aggregator.

Untuk pesan yang tidak dapat diproses, setiap consumer memiliki prefiks subject dead letter tersendiri pada stream HAZARDS_DLQ. Pesan yang memenuhi kriteria karantina diterbitkan ke subject DLQ yang dibentuk dari prefiks consumer, nama durable consumer, nama stream asal, dan nomor urut pesan. Stream HAZARDS_DLQ menyimpan maksimal satu pesan per subject, sehingga pengiriman ulang pesan yang sama tidak menghasilkan entri duplikat selama entri sebelumnya masih tersimpan. Pesan yang rusak dipindahkan ke stream tersebut agar tidak menghambat pemrosesan pesan berikutnya.

Alasan Pemilihan dan Trade-Off

Alternatif *broker* yang dibandingkan adalah Kafka, RabbitMQ, dan NATS JetStream. Ketiganya mendukung semantik pengiriman *at-least-once* dan *message replay*. Kafka merupakan yang paling berat untuk dijalankan pada lingkungan lokal dan memerlukan konfigurasi paling banyak. RabbitMQ berada di antara keduanya. NATS merupakan yang paling ringan karena cukup dijalankan dengan satu *binary* dan satu berkas konfigurasi, serta JetStream telah menyediakan *persistent stream*, *durable consumer*, dan *explicit acknowledgement*. Dengan laju *event* yang relatif rendah pada M1, kemampuan *partitioning* yang dimiliki Kafka tidak dibutuhkan.

Pada sisi penerbit, alternatif yang dipertimbangkan adalah menerbitkan *event* langsung ke *broker* setelah data disimpan ke basis data. Pendekatan ini lebih sederhana, tetapi memiliki celah. Apabila Aggregator berhenti di antara proses penyimpanan dan penerbitan, *event* akan tersimpan tetapi tidak pernah diterbitkan. Karena mekanisme deduplikasi mencegah *record* yang sama diterbitkan ulang pada *polling* berikutnya, *event* tersebut akan hilang secara permanen bagi *consumer*. *Outbox pattern* menutup celah tersebut dengan kompensasi berupa satu tabel tambahan dan satu proses *publisher*.

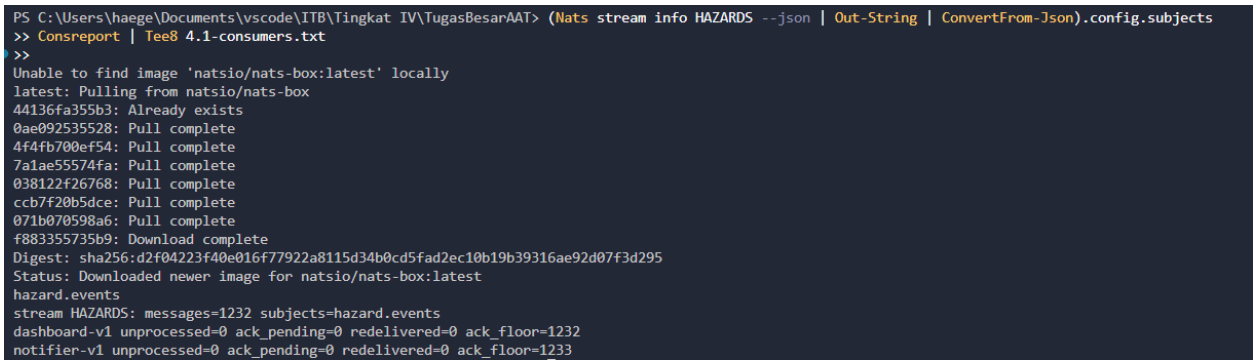
Semantik pengiriman yang dipilih adalah *at-least-once*. Pada sistem kebencanaan, peringatan yang hilang tanpa terdeteksi jauh lebih merugikan dibandingkan peringatan yang diterima dua kali. Konsekuensinya, *consumer* harus mampu menangani penerimaan pesan yang sama lebih dari satu kali.

Batasan Solusi

1. *Consumer* yang tidak aktif melebihi masa simpan *stream* tidak dapat memperoleh kembali pesan yang telah dihapus.
2. Pesan pada *dead letter queue* harus diproses ulang secara manual.

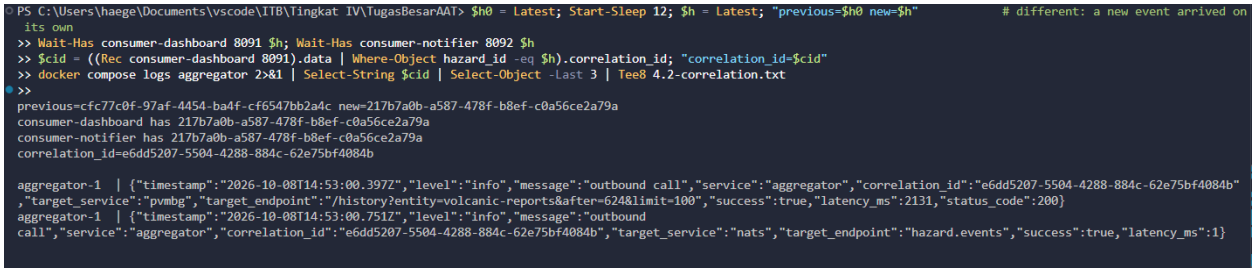
3. *Consumer* yang berhenti akibat kesalahan kritis, misalnya skema yang tidak didukung, dipulihkan setelah penyebab kegagalan diperbaiki. Operator meminta pemulihan melalui perintah lokal *consumer resume* yang dijalankan menggunakan docker compose exec pada container *consumer* terkait. Tidak tersedia *endpoint* HTTP maupun *scope client* untuk pemulihan.
4. Sistem tidak menjamin urutan pemrosesan pesan antarstream maupun saat pengiriman ulang, sehingga koreksi dapat diterima sebelum *event* awal. Untuk mencegah data lama menimpa data terbaru, *consumer* memperbarui proyeksi hanya ketika *revision* yang diterima lebih tinggi daripada *revision* yang tersimpan.
5. *Broker* dijalankan sebagai satu *instance*.

Pengujian dan Bukti Penyelesaian

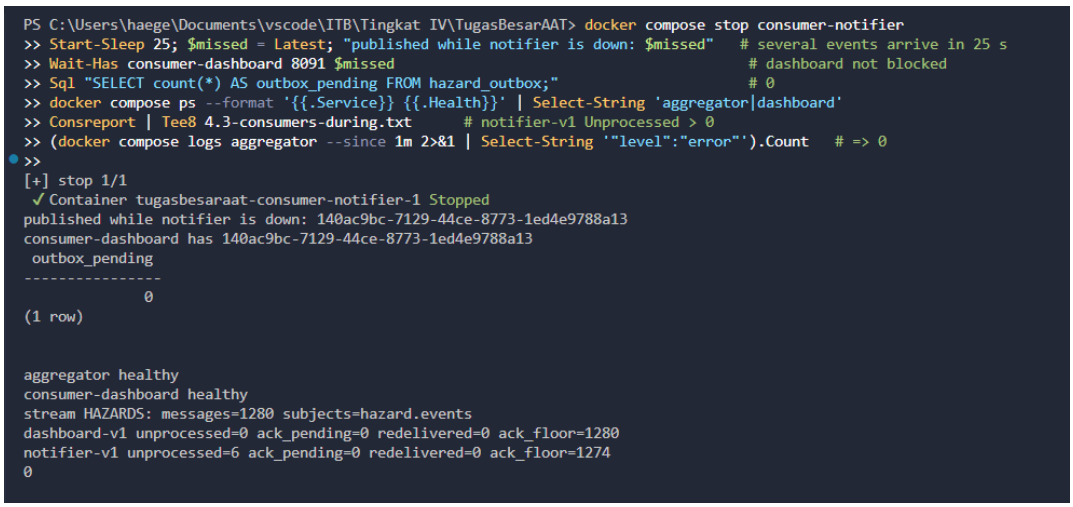


Gambar P5.1. Stream HAZARDS dan dua durable consumer independen.

Pengujian distribusi event diawali dengan memeriksa stream HAZARDS dan subscription yang tersedia. Gambar P5.1 menunjukkan subject hazard.events serta dua durable consumer, yaitu dashboard-v1 dan notifier-v1, dengan posisi pemrosesan masing-masing. Selanjutnya, satu HazardEvent baru diamati pada kedua consumer. Gambar P5.2 memperlihatkan hazard_id yang sama pada consumer dashboard dan notifier, disertai log Aggregator yang mencatat publikasi ke broker pada subject hazard.events. Hasil ini menunjukkan bahwa kedua consumer menerima event yang sama melalui jalur distribusi broker.

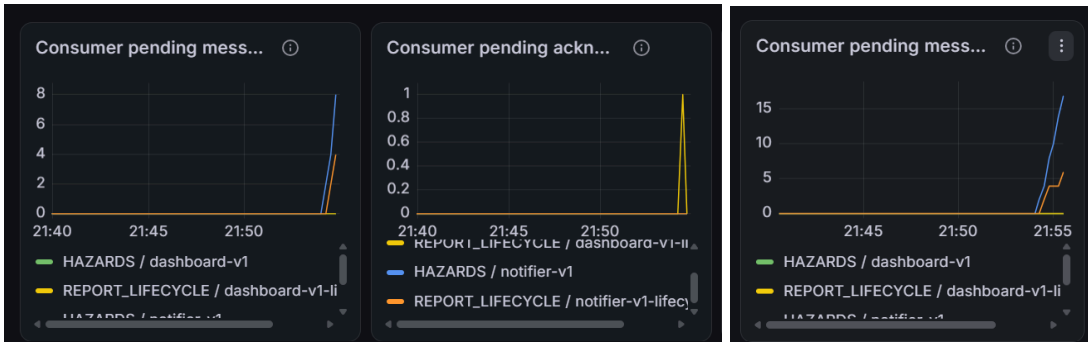


Gambar P5.2. Penerimaan HazardEvent yang sama oleh dashboard dan notifier.

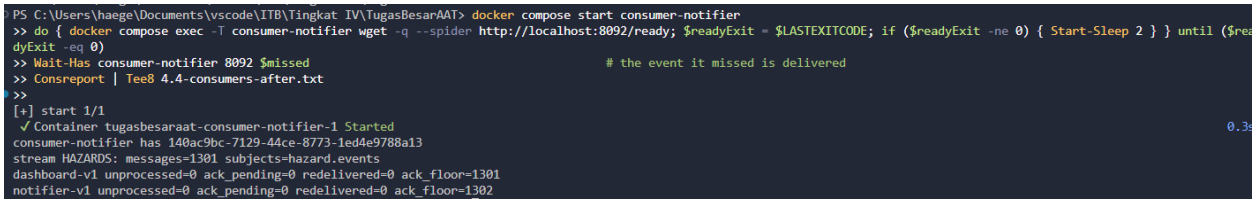


Gambar P5.3. Publikasi dan konsumsi event selama notifier tidak aktif.

Independensi consumer diuji dengan menghentikan consumer notifier. Pada Gambar P5.3, HazardEvent baru tetap dipublikasikan dan diterima oleh consumer dashboard ketika notifier tidak aktif. Jumlah pesan stream meningkat dari 1.232 pada pemeriksaan awal menjadi 1.280, sementara notifier memiliki enam pesan yang belum diproses. Pada saat yang sama, outbox_pending bernilai nol, Aggregator dan consumer dashboard berstatus healthy, serta pemeriksaan log Aggregator pada interval tersebut tidak menemukan entri error. Hasil tersebut menunjukkan bahwa penghentian notifier tidak menahan publikasi maupun penerimaan event oleh consumer lain.

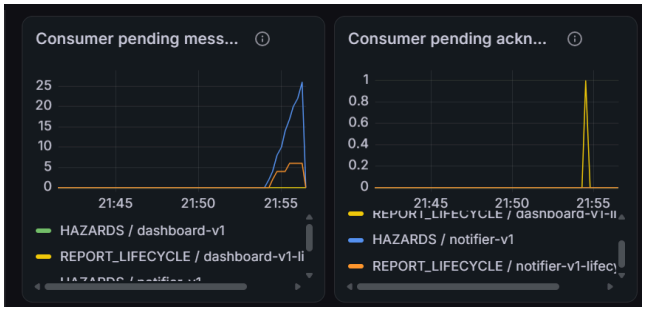


Gambar P5.4. Metrik pesan pending dan acknowledgement selama pengujian.



Gambar P5.5. Penerimaan event tertinggal setelah notifier dijalankan kembali.

Grafik pada Gambar P5.4 digunakan untuk mengamati perubahan metrik pesan pending dan pesan yang menunggu acknowledgement. Setelah notifier dijalankan kembali, Gambar P5.5 menunjukkan bahwa consumer tersebut menerima hazard_id yang diterbitkan ketika dirinya tidak aktif dan nilai unprocessed kembali menjadi nol. Bukti ini menunjukkan bahwa pesan yang tertinggal dapat diterima setelah pemulihan consumer selama masih tersedia dalam retensi stream. Grafik pada Gambar P5.6 menjadi pendukung pemantauan metrik; kesimpulan pemulihan backlog terutama didasarkan pada keluaran terminal.



Gambar P5.6. Pemantauan metrik consumer di sekitar periode pemulihan.

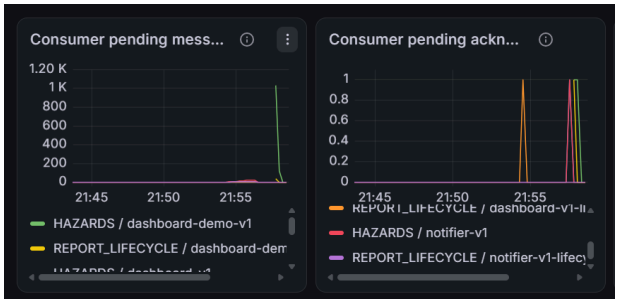
```
PS C:\Users\haege\Documents\vscode\ITB\Tingkat IV\TugasBesarAAT> Started | Set-Content evidence/4.5-started-before.txt -Encoding utf8
>> # Pre-flight: remove any demo container/volume left over from an earlier run (see the warning below)
>> docker compose --profile consumer-demo rm -sf consumer-dashboard-demo
>> docker volume ls -q -f name=dashboard-demo-data | ForEach-Object { docker volume rm $_ }
>> docker compose --profile consumer-demo up -d --force-recreate consumer-dashboard-demo
>> for ($i = 0; $i -lt 30; $i++) {                                  # bounded wait, max ~60 s
>>   $st = 'not_ready'; docker compose exec -T consumer-dashboard-demo wget -q --spider http://localhost:8093/ready
>>   if ($LASTEXITCODE -eq 0) { $st = 'ready'; break }; Start-Sleep 2 }
>> if ($st -ne 'ready') { "demo consumer NOT ready ($st):"; docker compose --profile consumer-demo logs --tail 5 consumer-dashboard-demo }
>> Start-Sleep 12; $h = Latest; "event published after the third consumer joined: $h"
>> foreach ($s in @(@('consumer-dashboard',8091),@('consumer-notifier',8092),@('consumer-dashboard-demo',8093))) { Wait-Has $s[0] $s[1] $h }   # all three have it
>> "demo consumer records: $((Rec consumer-dashboard-demo 8093).count)"                                          # history replayed
>> Started | Select-String aggregator; Select-String aggregator evidence/4.5-started-before.txt                  # same StartedAt
>> Consreport
>>
No stopped containers
tugasbesaraat_dashboard-demo-data
[+] Building 6.8s (17/17) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 707B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 397B
```

```
 => => exporting layers
 => => exporting manifest sha256:4abec590b7c1a0ce3fe4c8e46857dcf699d1b9b84460aa95da941910de678d73
 => => exporting config sha256:d5b6e3ee967ed0c761539dde71aa04f52e76b511bdb32529ee152d8c211bf274
 => => exporting attestation manifest sha256:d5b0b6ec88d0c5e3c8f6ae80cef3f13d800cb9788751f2f7143754c7c9a1f163
 => => exporting manifest list sha256:89668a848d8d24a904a0d476b04f88f66f82c840037588613c90529edb07ea95
 => => naming to docker.io/library/tugasbesaraat-consumer-dashboard-demo:latest
 => => unpacking to docker.io/library/tugasbesaraat-consumer-dashboard-demo:latest
 => resolving provenance for metadata file
[+] up 3/3
 ✓ Image tugasbesaraat-consumer-dashboard-demo       Built
 ✓ Volume tugasbesaraat_dashboard-demo-data          Created
 ✓ Container tugasbesaraat-consumer-dashboard-demo-1 Started
event published after the third consumer joined: 84a3dc5c-be3e-41dc-9ae9-8e984da31678
consumer-dashboard has 84a3dc5c-be3e-41dc-9ae9-8e984da31678
consumer-notifier has 84a3dc5c-be3e-41dc-9ae9-8e984da31678
consumer-dashboard-demo has 84a3dc5c-be3e-41dc-9ae9-8e984da31678
demo consumer records: 100

/tugasbesaraat-aggregator-1 2026-10-08T14:11:51.194827098Z
evidence\4.5-started-before.txt:1:/tugasbesaraat-aggregator-1 2026-10-08T14:11:51.194827098Z
stream HAZARDS: messages=1322 subjects=hazard.events
dashboard-v1 unprocessed=0 ack_pending=0 redelivered=0 ack_floor=1322
notifier-v1 unprocessed=0 ack_pending=0 redelivered=0 ack_floor=1322
dashboard-demo-v1 unprocessed=0 ack_pending=0 redelivered=0 ack_floor=1323
```

Gambar P5.7. Penambahan consumer ketiga dan penerimaan event pada subject yang sama.

Pengujian terakhir menambahkan consumer dashboard-demo pada subject yang sudah digunakan. Gambar P5.7 memperlihatkan container tambahan dijalankan, menerima hazard_id yang sama dengan dua consumer sebelumnya, dan memiliki durable subscription dashboard-demo-v1. Waktu mulai Aggregator tetap sama pada pemeriksaan sebelum dan sesudah penambahan consumer. Grafik pada Gambar P5.8 menunjukkan lonjakan pesan pending sekitar seribu pesan yang kemudian turun mendekati nol, konsisten dengan pemrosesan riwayat stream oleh consumer baru. Penambahan pelanggan dilakukan melalui konfigurasi consumer tanpa perubahan producer pada rangkaian pengujian yang ditampilkan.



Gambar P5.8. Pemrosesan backlog stream oleh consumer tambahan.

Pemilihan Broker, Semantik Pengiriman, dan Idempotensi Consumer

Broker yang dipilih adalah NATS JetStream. Pola yang digunakan adalah *publish-subscribe* dengan *stream* persisten untuk *event* kanonik dan *lifecycle*. Setiap pelanggan memiliki *durable subscription* tersendiri pada masing-masing jalur, sedangkan penerbit menggunakan *outbox* untuk mempertahankan pekerjaan publikasi ketika broker belum tersedia.

Semantik pengiriman yang diterapkan adalah *at-least-once*. Jaminan ini dibangun pada dua titik. Antara Aggregator dan *broker*, baris *outbox* baru dihapus setelah *broker* memberikan konfirmasi. Antara *broker* dan *consumer*, pesan baru dianggap selesai diproses setelah *consumer* mengirimkan *ack*, dan *ack* tersebut baru dikirimkan setelah data berhasil disimpan.

Konsekuensinya, pesan yang sama dapat diterima lebih dari satu kali. Kondisi ini dapat terjadi, misalnya, ketika Aggregator berhenti tepat setelah *broker* memberikan konfirmasi tetapi sebelum baris *outbox* dihapus, atau ketika *consumer* berhenti setelah menyimpan data tetapi sebelum *ack* dikirimkan. Oleh karena itu, *consumer* harus bersifat idempoten.

Penanganan duplikasi dilakukan melalui tiga lapisan berikut.

1. Aggregator menggunakan hazard_id sebagai *message ID* untuk HazardEvent baru dan event_id untuk *event lifecycle*. Berdasarkan *message ID* tersebut, *broker* membuang publikasi duplikat dalam *deduplication window*-nya. *Consumer* menangani duplikasi secara terpisah dengan mencatat identitas pesan berdasarkan *stream*, *sequence number*, dan *hash payload*, serta menggunakan event_id dan *revision* untuk mencegah pemrosesan *lifecycle* berulang.
2. Setiap *consumer* menyimpan identitas pesan, yaitu nama *stream*, nomor urut (*sequence number*), dan *hash* isi pesan, dalam transaksi SQLite yang sama dengan hasil pemrosesannya. Pesan dengan identitas yang telah tercatat tidak diproses ulang dan hanya dikirimkan *ack* kembali.
3. hazard_id ditetapkan sebagai *unique key* pada tabel hasil di setiap *consumer*, sehingga *event* yang sama tidak mungkin tersimpan dua kali meskipun diterima dengan nomor urut yang berbeda.

Event yang diterbitkan ketika suatu *consumer* tidak aktif tetap dapat diterima setelah *consumer* tersebut kembali berjalan. *Broker* mempertahankan pesan di dalam *stream* dan menyimpan posisi baca terakhir dari *durable consumer* tersebut. Ketika *consumer* dijalankan kembali, *consumer* melanjutkan pembacaan dari posisi tersebut dan menerima seluruh pesan yang terlewat, selama pesan masih berada dalam masa simpannya. Selama *consumer* tersebut tidak aktif, Aggregator tetap menerbitkan *event* dan

consumer lain tetap menerimanya seperti biasa, karena setiap *consumer* memiliki posisi baca masing-masing.

LAMPIRAN

Nama	NIM	Implementasi
Haegen Quinston	13523109	Mock BMKG, Mock PVMBG, Aggregator, Canonical Store
Steven Owen Liauw	13523103	Client API, Auth Service, Client TUI
Daniel Pedrosa Wu	13523099	Broker, Schema Management, Consumers, Monitoring

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM :

- Daniel Pedrosa Wu / 13523099
- Steven Owen / 13523103
- Haegen Quinston / 13523109

Fakultas/Sekolah : STEI-K

Kode Mata Kuliah : IF4031

Nama Mata Kuliah : Arsitektur Aplikasi Terdistribusi

Judul Tugas/Proposal : *Milestone* 1 Tugas Besar

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah: ChatGPT, Claude

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	Tidak Digunakan
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Sedang
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Rendah
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	Tidak Digunakan-
Lainnya, Jelaskan:	Tidak	Tidak Digunakan

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 05-10-2026



Daniel Pedrosa Wu

13523099



Steven Owen Liauw

13523103



Haegen Quinston

13523109